

Human-in-the-Loop Reinforcement Learning on a Real Robot Arm

Theory, Mathematics and Practice with LeRobot and the SO-101

Lucas Rodrigues Pereira

LeRobot v0.6.1 (commit 2aba372b)

2 August 2026

Contents

Front Matter

- Preface
- Notation
- How to verify your setup

Back matter

Chapter 1: The Problem and the Formalism

- 1.1 Why manipulation resists direct programming
- 1.2 The MDP
- 1.3 Return, value functions, and the Bellman equations
- 1.4 Manipulation from pixels is a POMDP
- 1.5 Exploration, and why sparse reward is hopeless from scratch
- 1.6 On-policy versus off-policy, and the real constraint
- 1.7 What HIL-SERL combines

Chapter 2: Soft Actor-Critic

- 2.1 The maximum-entropy objective
- 2.2 Soft value functions
- 2.3 The soft Bellman backup operator and its contraction
- 2.4 Soft policy evaluation and soft policy improvement
- 2.5 The practical algorithm
- 2.6 The three losses, explicitly
- 2.7 Symbol to config key
- 2.8 Two undocumented divergences from the reference implementation

Chapter 3: Learning from Prior Data (RLPD) and the Replay Buffer

- 3.1 The offline-to-online problem
- 3.2 Why pretrain-then-finetune fails
- 3.3 RLPD's three ingredients
- 3.4 Update-to-data ratio and what it costs
- 3.5 Why the de-tuned defaults are defensible
- 3.6 The ReplayBuffer
- 3.7 The float32 image trap, with arithmetic

Chapter 4: Human-in-the-Loop and the Reward Problem

- Part A: The Intervention Mechanism
- Part B: The Reward Problem

Chapter 5: Kinematics and the End-Effector Action Space

- 5.1 Forward kinematics as a product of homogeneous transforms
- 5.2 The manipulator Jacobian
- 5.3 Inverse kinematics, damping, and singularities
- 5.4 Why HIL-SERL learns in end-effector delta space
- 5.5 What LeRobot actually implements: 3-DoF translation
- 5.6 The processor chain
- 5.7 Bounds and step sizes: the highest-leverage knob you have

Chapter 6: The Simulation Practicum

- 6.1 MuJoCo and what gym_hil adds on top
- 6.2 The registered environments, and the trap in PandaPickCubeBase-v0
- 6.3 The observation and the reward

- 6.4 Actor and learner: two processes, one gRPC socket
- 6.5 Launching the run
- 6.6 What to watch in WandB, in order
- 6.7 Why this worked, and why none of it transfers
- 6.8 Troubleshooting two failures you will hit again

Chapter 7: The Real Robot Pipeline

- Stage A: Hardware bring-up and device stability
- Stage B: URDF and workspace bounds
- Stage C: Demonstrations
- Stage D: Crop the ROI and freeze the visual world
- Stage E: Reward
- Stage F: Training
- Stage G: Evaluation
- Expectations

Chapter 8: Hyperparameters, Failure Modes and Experimental Practice

- Part A: Hyperparameter reference
- Part B: Failure-mode register, ranked
- Part C: Experimental practice

Further Reading

- The four papers this book is built on
- Useful supporting papers
- Documentation and code
- Reading order suggestion

Front Matter

Preface

This book has a single, narrow, stubborn goal: to take a reader who cannot currently define a Q-function all the way to a physical SO-101 arm that improves at a manipulation task while a human holds a gamepad and corrects it. Not a video of someone else's robot. Yours, on your desk, with your two USB cameras and your servo calibration file.

Who this is for. Graduate students and research engineers in robotics or machine learning who write Python comfortably, can read a matrix expression without flinching, and have a Linux box with a decent GPU. It is written for the person who has to defend the system in a viva, which means every design choice gets a reason and every equation gets a derivation, not a citation.

What it assumes. Python and NumPy. Linear algebra: matrix products, rank, homogeneous transforms. Basic probability: expectation, conditional distributions, and enough comfort to accept a KL divergence when it is defined for you. Gradients and the chain rule. Git, a terminal, and the patience to read a stack trace.

What it does not assume. Any reinforcement learning at all. Markov decision processes, the Bellman backup, policy gradients, actor-critic architectures, entropy regularisation and the reparameterisation trick are all built from nothing in Chapters 1 to 5.

What it does not cover. On-policy methods (PPO, TRPO) appear only as a contrast used to explain why they are hopeless at 10 Hz on real hardware. Model-based RL, offline-only RL, diffusion and vision-language-action policies, sim-to-real domain randomisation, ROS 2 integration, the mechanical assembly and soldering of the SO-101 itself, multi-GPU scaling, and formal convergence proofs are all out of scope. This is one algorithm family, done properly, on one robot.

How to read it. Chapters 1 to 5 are theory: the MDP, the Bellman equations and why off-policy learning is mandatory when every sample costs robot time (1); soft actor-critic derived from the maximum-entropy objective, with every symbol mapped to a `configuration_sac.py` key (2); RLPD-style offline-online mixing and the replay buffer engineering that decides whether your run fits in 24 GB (3); the human-in-the-loop mechanism and the reward problem, including learned success classifiers (4); and the kinematics that make end-effector action spaces possible (5). Chapters 6 to 8 are practice: the simulated pick task with `gym-hil`, which you can run today (6); the real SO-101 pipeline stage by stage, from udev rules to a training run (7); and a hyperparameter reference, a ranked failure-mode catalogue and notes on experimental practice for a thesis (8).

Read it linearly, but start Chapter 6 in a terminal while you are still reading Chapter 2. The simulated loop reaches a working state in an afternoon and gives you a live, logging, occasionally-succeeding artefact to attach the mathematics to. Mathematics without a running process in the next window is much harder to hold onto.

An honest word about drift. Every number, path and line reference in this book was produced by running the code, not by reading about it. That code is LeRobot v0.6.1, commit `2aba372b`, on Ubuntu 24.04.4 with Python 3.12.3 and torch 2.11.0+cu128, in August 2026. Two of the findings here are defects in that exact tree: `src/lerobot/envs/configs.py:284` annotated `fixed_reset_joint_positions` as `Any | None`, which draccus cannot encode, crashing `learner.py:166` (fixed in PR #4297), and `S0Leader` on v0.6.1 has no `get_teleop_events()`, so leader-arm interventions are impossible (PR #3086 open). Both will eventually be fixed upstream and those passages will become history. The mathematics in Chapters 1 to 5 will not drift. The file paths, line numbers and config keys in Chapters 6 to 8 certainly will. Treat line numbers as hints and grep for the symbol name.

Notation

Symbols are consistent across all chapters. Where a symbol corresponds to something you can actually set, the config key or source location is given, because the fastest way to make an equation concrete is to find the line that implements it.

Markov decision process

Symbol	Meaning	In the code
$\mathcal{S}$	State space	
s_t	State at time t . In practice a dict, not a vector	
o_t	Observation: <code>observation.images.front</code> , <code>observation.images.wrist</code> (3x128x128) and <code>observation.state</code> (18-dim in sim)	<code>gym_manipulator.py</code>
$\mathcal{A}$	Action space. Here $\mathbb{R}^3$ end-effector deltas plus one discrete gripper symbol	
a_t	Action at time t , $(\Delta x, \Delta y, \Delta z, g)$	<code>hil_processor.py</code>
g	Discrete gripper command, <code>num_discrete_actions=3</code> (open / close / stay)	<code>configuration_sac.py</code>
$r_t = r(s_t, a_t)$	Scalar reward	<code>panda_pick_gym_env.py:_compute_reward</code>
$\gamma \in [0, 1)$	Discount factor, 0.97 in the reference config	<code>discount</code>
$P(s' s, a)$	Transition kernel. Never modelled, only sampled	
H	Episode horizon in steps. At <code>fps: 10</code> , a 10 s episode is 100 steps	
ζ	A trajectory $(s_0, a_0, r_0, s_1, \dots)$. Note: many papers write τ for this. We reserve τ for the Polyak coefficient	
$\rho_\pi(s)$	State visitation distribution induced by π	

Policies, values and the soft objective

Symbol	Meaning	In the code
$\pi_\phi(a s)$	Stochastic policy (tanh-squashed Gaussian) with parameters ϕ	<code>policies/gaussian_actor/</code> , <code>policy.type: "gaussian_actor"</code>
$\mu_\phi(s), \sigma_\phi(s)$	Pre-squash mean and standard deviation output by the actor head	
$\epsilon \sim \mathcal{N}(0, I)$	Reparameterisation noise, $a = \tanh(\mu_\phi(s) + \sigma_\phi(s) \odot \epsilon)$	
$V^\pi(s)$	Soft state value	
$Q^\pi(s, a)$	Soft action value	
Q_{θ_i}	i -th critic, $i = 1 \dots N, N = 2$	<code>num_critics</code>
$\theta, \bar{\theta}$	Critic parameters and target-network parameters	
τ	Polyak coefficient in $\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta}$, default 0.005	<code>critic_target_update_weight</code>
$\mathcal{H}(\pi(\cdot s)) = -\mathbb{E}_{a \sim \pi}[\log \pi(a s)]$	Policy entropy at state s	
α	Entropy temperature, the exchange rate between reward and entropy. Initialised at 0.01, then learned	<code>temperature_init</code> , <code>log_alpha</code>
$\bar{\mathcal{H}}$	Target entropy. Defaults to $-\dim(\mathcal{A})/2$	<code>sac_algorithm.py:125</code>
$J(\pi)$	The max-entropy objective $\sum_t \mathbb{E}[r_t + \alpha \mathcal{H}(\pi(\cdot s_t))]$	

Symbol	Meaning	In the code
$\mathcal{T}^\pi$	Soft Bellman operator	
y	TD target $r + \gamma(\min_i Q_{\delta_i}(s', a') - \alpha \log \pi_\phi(a' s'))$. The entropy term is optional	<code>use_backup_entropy</code>
c	Gradient-norm clip threshold, 10.0	<code>grad_clip_norm</code>

Note carefully that J is overloaded in the robotics literature. In this book $J(\pi)$ with a policy argument is always the RL objective, and $J(q)$ with a joint-configuration argument is always the Jacobian. They never appear in the same equation.

Data, buffers and the human

Symbol	Meaning	In the code
$\mathcal{D}$	Replay buffer, a set of transitions (s, a, r, s', d)	<code>buffer.py</code>
$\mathcal{D}_{\text{off}}$	Offline buffer, seeded from human demonstrations (458 transitions in the sim example)	<code>offline_buffer_capacity: 100000</code>
$\mathcal{D}_{\text{on}}$	Online buffer, filled by the actor	<code>online_buffer_capacity: 100000</code>
β	Fraction of each minibatch drawn from the online buffer, 0.5	<code>online_ratio</code> , <code>mixer: "online_offline"</code>
B	Minibatch size	<code>batch_size</code>
G	Update-to-data ratio: gradient steps per environment step, 2	<code>utd_ratio</code>
$d_t \in \{0, 1\}$	Terminal / done flag	
π_H	The human's (implicit) policy, expressed through the gamepad	
$\mathbb{1}_t^H$	Intervention indicator: 1 when the human overrode the actor at step t	<code>hil_processor.py</code>
a_t^H	The action the human supplied, which replaces a_t in the stored transition	

Rewards and classifiers

Symbol	Meaning	In the code
$R_\psi(o)$	Learned reward classifier with parameters ψ , a sigmoid over an image encoder	<code>modeling_classifier.py</code>
z	Height of the manipulated object; z_{init} its height at reset	<code>block_pos</code> sensor
$\text{lift} = z - z_{\text{init}}$	Sparse sim success is $\mathbb{1}[\text{lift} > 0.1]$	<code>_compute_reward</code>
ψ_{thr}	Decision threshold on the classifier logit	<code>configuration_classifier.py</code>

Kinematics and the robot

Symbol	Meaning	In the code
$q \in \mathbb{R}^n$	Joint configuration (6 actuated joints on the SO-101 follower)	
$\dot{q}$	Joint velocities	
$x = f(q) \in SE(3)$	Forward kinematics: end-effector pose	<code>model/kinematics.py</code>
$J(q) = \partial f / \partial q$	Manipulator Jacobian mapping $\dot{q} \mapsto$ end-effector twist	<code>placo.RobotWrapper(urdf_path)</code>
$J^\dagger$	Pseudo-inverse (or damped least squares) used to turn a commanded Δx into Δq	

Symbol	Meaning	In the code
Δx	Commanded end-effector displacement per control step, the actual action	<code>InverseKinematicsConfig</code>
$\Delta t = 1/\text{fps}$	Control period. <code>fps: 10</code> gives $\Delta t = 0.1$ s	

How to verify your setup

Four commands. Run them inside the project virtualenv before you run anything else, and rerun them after any upgrade to torch, CUDA or LeRobot. Each gate isolates one subsystem that has historically broken independently of the others, so a failure tells you exactly which one to fix.

```
# Gate 1: IK backend. The one that bites on Ubuntu 24.04 (LeRobot issue #3755).
python -c "import placo; print('placo ok')"

# Gate 2: GPU. Must print a CUDA build and True, not just a version.
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"

# Gate 3: Video decoding. Dataset loading dies here, not at import time.
python -c "from torchcodec.decoders import VideoDecoder; print('decoder ok')"

# Gate 4: Hardware and IPC. gRPC (actor/learner), pygame (gamepad), Feetech SDK (servos).
python -c "import grpc, pygame, feetech_servo_sdk; print('hw ok')"
```

Expected output on the reference machine:

```
placo ok
2.11.0+cu128 True
decoder ok
hw ok
```

Gate	Protects	Typical failure
1	<code>src/lerobot/model/kinematics.py</code> , all end-effector control	Missing or ABI-mismatched <code>urdfdom</code> / <code>tinymx12</code> . The <code>cmeel-urdfdom</code> and <code>cmeel-tinymx12</code> pins in v0.6.1 exist precisely to prevent this
2	The learner process, every gradient step	CPU-only wheel installed, or driver older than the CUDA build
3	Loading the offline demonstration dataset	Codec or NVIDIA NPP libraries absent. Note that ffmpeg version is usually not the culprit
4	Actor-learner gRPC channel, gamepad interventions, servo bus	Missing extras, or <code>pygame</code> present but no joystick device enumerated

Install with the right extras. Use `.[hilserl,feetech,gamepad,core_scripts,training]`. Do **not** use `.[all]`: it pulls `hf-libero`, which pulls `robomimic`, which pulls `egl-probe`, which builds from source and needs CMake. LIBERO is an imitation-learning benchmark with no bearing on HIL-SERL, so you would be paying a toolchain tax for nothing.

Keep your configs outside the submodule. Put JSON configs in a top-level `configs/` directory, not inside `vendor/lerobot`, so that pulling the submodule never touches your experiment definitions and `git status` stays readable.

One trap worth checking before your first long run. `ReplayBuffer._initialize_storage` in `src/lerobot/r1/buffer.py` allocates with `torch.empty(...)` and no `dtype`, which yields float32 image storage. With `online_buffer_capacity: 100000`, two cameras and 3x128x128 frames, that is far past 24 GB if `storage_device` is `cuda`. Keep `storage_device: cpu` unless you have reduced the capacity by an order of magnitude and done the arithmetic yourself.

Chapter 1: The Problem and the Formalism

1.1 Why manipulation resists direct programming

Write a program that picks up a cube. If the cube is bolted to a known fixture and the arm is well calibrated, this is a solved problem: measure the pose, run inverse kinematics, interpolate a trajectory, close the gripper. This is what industrial robotics has done since the 1970s, and it works because the environment has been engineered until the uncertainty is gone.

Remove the fixture and the approach falls apart in a way that is instructive. The failure is not that the geometry becomes hard. It is that the mapping from *what the robot senses* to *what the robot should do next* stops being expressible as a short sequence of conditionals. Contact is the main culprit. The dynamics of a gripper closing on a deformable object are discontinuous, high-dimensional and extremely sensitive to millimetre-scale errors in relative pose. A grasp that succeeds at 2 mm offset fails at 4 mm, and no amount of forward modelling tells you where that boundary lies for an object you have not held before. Perception compounds this: the pose estimate that feeds your IK solver has its own error distribution, and that error is correlated with exactly the visual conditions (occlusion by the gripper itself, specularity, clutter) that arise precisely when you are about to make contact.

There is also a subtler point. Even where a hand-written controller is possible, it encodes a *policy* that a human derived by trial and error, and the human's trials are slow and undocumented. Reinforcement learning proposes to automate that loop: specify what success means, let the system discover the mapping. The rest of this chapter makes "specify what success means" and "discover the mapping" precise, and then shows why the naive version of this proposal fails badly on real hardware.

1.2 The MDP

The standard formalism is the Markov Decision Process, a tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0).$$

$\mathcal{S}$ is the state space. For the simulated Franka pick task in `gym_hil/envs/panda_pick_gym_env.py` the underlying MuJoCo state includes joint angles, joint velocities, gripper aperture and the block pose; what the agent receives is a 18-dimensional `observation.state` vector (7 joint positions, 7 joint velocities, 1 gripper, 3 end-effector *xyz*) plus two $3 \times 128 \times 128$ camera images.

$\mathcal{A}$ is the action space. In the HIL-SERL configuration it is hybrid: three continuous Cartesian deltas ($\Delta x, \Delta y, \Delta z$) plus one discrete gripper command with `num_discrete_actions=3`, encoded close = 0, stay = 1, open = 2. These are assembled in `InterventionActionProcessorStep` in `src/lerobot/processor/hil_processor.py` from the teleoperator's `delta_x`, `delta_y`, `delta_z` fields.

$P(s' | s, a)$ is the transition kernel, a conditional probability distribution over next states. The *Markov property* is the assumption that this distribution depends on the history only through the current state:

$$\Pr(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} | s_t, a_t).$$

$r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward. In the sparse branch of `_compute_reward` it is brutally simple:

```
block_pos = self._data.sensor("block_pos").data
lift = block_pos[2] - self._z_init
return float(lift > 0.1)
```

One if the block has risen 10 cm, zero otherwise. The dense alternative, $0.3 \exp(-20d) + 0.7 \text{clip}\left(\frac{z - z_{\text{init}}}{z_{\text{succ}} - z_{\text{init}}}, 0, 1\right)$, shapes this into something continuously informative, at the cost of encoding a human guess about *how* the task should be solved.

$\gamma \in [0, 1)$ is the discount factor (0.97 in the reference config), and ρ_0 the initial state distribution, controlled here by `ResetConfig` in `src/lerobot/envs/configs.py`.

A policy $\pi(a \mid s)$ is a conditional distribution over actions. Together with P and ρ_0 it induces a distribution over trajectories $\tau = (s_0, a_0, s_1, a_1, \dots)$.

1.3 Return, value functions, and the Bellman equations

Define the discounted return from time t :

$$G_t = \sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}).$$

The discount does two jobs. Mathematically it guarantees $|G_t| \leq r_{\max}/(1 - \gamma) < \infty$, so the objective is well defined over infinite horizons. Practically it sets an effective horizon of roughly $1/(1 - \gamma)$ steps: at $\gamma = 0.97$ that is about 33 steps, which at `fps: 10` is 3.3 seconds of robot time. That number is a design choice, not a constant of nature, and it should be long enough to span the causal chain from "approach" to "reward".

The value functions are the expected return under π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].$$

Now derive the Bellman equation. Split the return into the immediate reward and the rest:

$$G_t = r(s_t, a_t) + \gamma \sum_{k=0}^{\infty} \gamma^k r(s_{t+1+k}, a_{t+1+k}) = r(s_t, a_t) + \gamma G_{t+1}.$$

Take the conditional expectation given $s_t = s, a_t = a$. The immediate reward is deterministic given (s, a) . For the second term, condition on s_{t+1} and use the tower property:

$$\mathbb{E}_\pi[G_{t+1} \mid s_t = s, a_t = a] = \mathbb{E}_{s' \sim P(\cdot \mid s, a)} [\mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s']] = \mathbb{E}_{s' \sim P} [V^\pi(s')],$$

where the inner equality is exactly the Markov property: once s_{t+1} is known, the past is irrelevant. Hence

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot \mid s, a)} [V^\pi(s')].$$

And since $V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot \mid s)} [Q^\pi(s, a)]$, substituting gives the two coupled equations

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [r(s, a) + \gamma \mathbb{E}_{s' \sim P} [V^\pi(s')]], \quad Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P} \mathbb{E}_{a' \sim \pi} [Q^\pi(s', a')].$$

The optimal value functions satisfy the Bellman *optimality* equations, where the expectation over π is replaced by a maximum:

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P} \left[\max_{a'} Q^*(s', a') \right], \quad \pi^*(s) \in \arg \max_a Q^*(s, a).$$

Both Bellman operators are γ -contractions in the sup norm, which is why value iteration converges. Everything in `sac_algorithm.py` is a stochastic, function-approximated version of this backup, with one modification (an entropy term) that Chapter 3 derives.

1.4 Manipulation from pixels is a POMDP

The Markov property is an assumption about *the state*, and a camera image is not the state. Velocity is not visible in a single frame. Occluded object geometry is not visible. Whether the gripper has actually achieved force closure is not visible. Formally the agent faces a Partially Observable MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma, \Omega, O)$ with observations $o \sim O(\cdot \mid s)$, and the optimal policy is a function of the belief $b_t = \Pr(s_t \mid o_{0:t}, a_{0:t-1})$, not of o_t .

Two cheap approximations recover most of what matters. **State augmentation**: concatenate proprioception to the images, which is why `observation.state` carries joint *velocities* alongside positions and end-effector *xyz*. Velocity is precisely the quantity a single frame cannot supply. **Frame stacking**: feed the last k frames as one

observation, making finite-history information available so that quantities with bounded temporal support become recoverable. Neither gives a true belief state; both make the residual non-Markovianity small enough that a Markov algorithm behaves well in practice. This is a pragmatic compromise, and it is worth being honest about it in a viva: the convergence guarantees of Section 1.3 do not formally survive it.

1.5 Exploration, and why sparse reward is hopeless from scratch

Consider the sparse reward above with a fresh policy. Actions are three-dimensional Gaussian deltas, effectively a random walk of the end-effector. To receive any non-zero reward the agent must, by chance: reach the block's xy position within grasp tolerance, descend, close the gripper at the right moment, and lift 10 cm while maintaining contact. The probability of a random 30-step trajectory doing this is astronomically small, and it must happen *at least once* before the reward function contains any information at all. Until then every gradient is on a constant-zero signal, and the policy has no reason to move toward the block rather than away.

This is not a tuning problem. It is structural: sparse reward defines the task but provides no gradient toward it, and the volume of the successful region in trajectory space is vanishing. The three standard escapes are reward shaping (which reintroduces human guesswork and can make the optimal policy wrong), simulation with massive parallelism (which reintroduces the sim-to-real gap), and *demonstrations*, which simply hand the agent successful trajectories so the reward is non-degenerate from step one. HIL-SERL takes the third route and pushes it further: the human stays in the loop.

1.6 On-policy versus off-policy, and the real constraint

An on-policy method (REINFORCE, PPO) estimates $\nabla_{\theta} J(\pi_{\theta})$ using trajectories drawn from π_{θ} itself. As soon as θ updates, the old data is off-distribution and must be discarded. An off-policy method learns Q from transitions (s, a, r, s') generated by *any* behaviour policy, because the Bellman backup in Section 1.3 conditions on a and never requires $a \sim \pi$; only the bootstrap term $a' \sim \pi$ does, and that is sampled from the current policy at update time, not from the stored data.

The consequence is decisive. On real hardware every transition costs wall-clock time, at `fps: 10`, plus resets, plus wear. There is no budget for throwing data away. Off-policy learning permits a replay buffer (`src/lerobot/rl/buffer.py`), which permits reusing each transition many times (`utd_ratio: 2` means two gradient steps per environment step), and it permits mixing sources: the reference config uses `mixer: "online_offline"` with `online_ratio: 0.5`, so half of every batch comes from a fixed offline demonstration set (in sim, `lilkm/pick_cube_franka_panda_30`, 458 transitions) and half from freshly collected online data. On-policy RL cannot do this at all. **Sample efficiency is the binding constraint in real-robot RL**, and off-policy learning with replay is not a preference, it is a prerequisite.

1.7 What HIL-SERL combines

Three ingredients, each answering one failure above:

1. **An off-policy maximum-entropy actor-critic (SAC)** with a learned Q , a replay buffer and an asynchronous actor/learner split (`src/lerobot/rl/actor.py`, `src/lerobot/rl/learner.py`), so the robot never waits on gradients and no transition is wasted.
2. **Human demonstrations plus human interventions**, seeding the buffer with successful trajectories and letting a gamepad operator take over mid-episode when the policy is about to fail, converting the impossible exploration problem of Section 1.5 into a tractable one.
3. **A learned reward classifier** (`src/lerobot/rewards/classifier/modeling_classifier.py`), which supplies the sparse success signal on real hardware where no `block_pos` sensor exists.

The remaining chapters take these apart in order.

Chapter 2: Soft Actor-Critic

Everything HIL-SERL does on top of standard RL (human interventions, demonstration seeding, a learned reward classifier) is layered onto one off-policy algorithm: Soft Actor-Critic. This chapter derives SAC properly, then maps every symbol onto the exact keys you will type into `configs/sim/train_config.json`.

2.1 The maximum-entropy objective

Standard RL maximises expected discounted return,

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right].$$

The optimal solution of this objective is generically a deterministic policy. That is a problem for a robot learning from scratch on real hardware: a deterministic policy collapses onto whatever mode it found first, stops exploring, and is brittle to the small perturbations that a real SO-101 will experience (a cube 5 mm to the left, a slightly different friction coefficient).

Maximum-entropy RL adds an entropy bonus at every timestep:

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} \left[\gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right], \quad \mathcal{H}(\pi(\cdot | s)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a | s)].$$

The temperature $\alpha > 0$ trades reward against stochasticity. As $\alpha \rightarrow 0$ you recover standard RL; as $\alpha \rightarrow \infty$ the policy becomes uniform.

Three consequences matter for us. First, exploration is built into the objective rather than bolted on as an ϵ -greedy or fixed-noise hack, and the amount of exploration is state-dependent: the policy stays broad where the critic is flat and sharpens where the critic has structure. Second, the optimal max-ent policy is a Boltzmann distribution over soft Q-values, so it assigns mass to *all* near-optimal modes rather than one, which is exactly the robustness property you want when a demonstration shows two viable grasp approaches. Third, and practically most important for HIL-SERL, entropy regularisation makes the critic learning problem better conditioned when the replay buffer mixes on-policy data with off-policy human interventions and offline demonstrations, because the backup targets are smoothed by an expectation rather than a hard max.

2.2 Soft value functions

Define the soft state value and soft action value under a policy π :

$$\begin{aligned} V^{\pi}(s) &= \mathbb{E}_{a \sim \pi} [Q^{\pi}(s, a) - \alpha \log \pi(a | s)], \\ Q^{\pi}(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim p} [V^{\pi}(s')]. \end{aligned}$$

Substituting one into the other gives the soft Bellman equation for Q^{π} :

$$Q^{\pi}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p, a' \sim \pi} [Q^{\pi}(s', a') - \alpha \log \pi(a' | s')]. \quad (2.1)$$

The only change from the ordinary Bellman equation is the $-\alpha \log \pi(a' | s')$ term inside the expectation. That term is the "backup entropy", and whether or not it is included is a config flag in LeRobot (`use_backup_entropy`), which we return to in §2.8.

2.3 The soft Bellman backup operator and its contraction

Define the operator $\mathcal{T}^{\pi}$ acting on any bounded function $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$:

$$(\mathcal{T}^{\pi}Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p} [V(s')], \quad V(s') = \mathbb{E}_{a' \sim \pi} [Q(s', a') - \alpha \log \pi(a' | s')].$$

Claim: $\mathcal{T}^\pi$ is a γ -contraction in the sup norm.

Proof. Take two functions Q_1, Q_2 . The reward and the entropy term are identical in both, so they cancel:

$$|(\mathcal{T}^\pi Q_1)(s, a) - (\mathcal{T}^\pi Q_2)(s, a)| = \gamma |\mathbb{E}_{s', a'} [Q_1(s', a') - Q_2(s', a')]| \leq \gamma \mathbb{E}_{s', a'} |Q_1 - Q_2| \leq \gamma \|Q_1 - Q_2\|_\infty.$$

Taking the sup over (s, a) gives $\|\mathcal{T}^\pi Q_1 - \mathcal{T}^\pi Q_2\|_\infty \leq \gamma \|Q_1 - Q_2\|_\infty$. Since $\gamma < 1$, Banach's fixed-point theorem applies. $\square$

The key observation is that the entropy term is a function of π and s' only, not of Q , so it is a constant offset from the operator's point of view. This is why max-ent RL inherits all the convergence machinery of ordinary RL for free.

2.4 Soft policy evaluation and soft policy improvement

Soft policy evaluation. For a fixed π with bounded reward and $|\mathcal{A}| < \infty$, the sequence $Q^{k+1} = \mathcal{T}^\pi Q^k$ converges to the unique fixed point Q^π , the soft Q-function of π . This is immediate from the contraction above.

Soft policy improvement. Given $Q^{\pi_{\text{old}}}$, update the policy by an information projection onto a tractable family Π :

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot | s) \parallel \frac{\exp(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(s, \cdot))}{Z^{\pi_{\text{old}}}(s)} \right). \quad (2.2)$$

The theorem states that $Q^{\pi_{\text{new}}}(s, a) \geq Q^{\pi_{\text{old}}}(s, a)$ for all (s, a) . The proof works because $\pi_{\text{old}} \in \Pi$ is always feasible, so the minimiser has KL no larger than π_{old} 's, which after expanding the KL yields $\mathbb{E}_{\pi_{\text{new}}} [Q - \alpha \log \pi_{\text{new}}] \geq V^{\pi_{\text{old}}}(s)$; repeatedly substituting this inequality into (2.1) telescopes into the claimed monotonicity.

Soft policy iteration. Alternating the two steps converges to a π^* optimal within Π . SAC is the function-approximation version: instead of running evaluation to convergence, do one gradient step on each, on minibatches drawn from a replay buffer.

Note what $\log Z(s)$ does in (2.2): it is independent of a , so it drops out of the gradient. This is what makes the actor loss in §2.5 implementable without ever computing a partition function.

2.5 The practical algorithm

Twin critics and min-of-two-Q

TD learning with a max (or a maximising actor) over a noisy Q estimate produces systematic *overestimation*, because $\mathbb{E}[\max_i X_i] \geq \max_i \mathbb{E}[X_i]$. Bootstrapping then compounds the bias. SAC borrows TD3's fix: maintain N independently initialised critics $Q_{\theta_1}, \dots, Q_{\theta_N}$ and use the pointwise minimum in the target,

$$y = r + \gamma(1 - d) \left[\min_i Q_{\bar{\theta}_i}(s', a') - \alpha \log \pi_\phi(a' | s') \right], \quad a' \sim \pi_\phi(\cdot | s'). \quad (2.3)$$

The min is a deliberately pessimistic estimator: it trades a controlled underestimation bias for the elimination of a divergent overestimation bias. In LeRobot this is `q_targets.min(dim=0)` in `sac_algorithm.py:303`, with $N = \text{num_critics}$.

Target networks and Polyak averaging

The bootstrap target (2.3) uses lagged parameters $\bar{\theta}$, updated by an exponential moving average,

$$\bar{\theta} \leftarrow \tau \theta + (1 - \tau) \bar{\theta}, \quad \tau \ll 1,$$

which is exactly `_update_target_networks` in `sac_algorithm.py:422`. The effective averaging horizon is $\approx 1/\tau$ updates, so $\tau = 0.005$ means the target trails by roughly 200 gradient steps. This decouples the

regression target from the regressed function and is what keeps the fixed-point iteration stable under function approximation.

Two implementation details worth knowing before a viva. First, `_update_target_networks` is called once per inner UTD iteration *and* once at the end of `update()`, so with `utd_ratio: 2` the targets are Polyak-updated twice per optimisation step and the effective time constant halves. Second, `critic_target` is constructed with the *same* encoder object as `critic_ensemble` (`sac_algorithm.py:90`); only the MLP heads are genuinely lagged. With `freeze_vision_encoder: true` this is harmless, since the encoder is not learning anyway, but it is a real divergence from a textbook target network.

Reparameterisation and the tanh log-det Jacobian

The actor loss requires $\nabla_{\phi} \mathbb{E}_{a \sim \pi_{\phi}}[\cdot]$ where the distribution itself depends on ϕ . A score-function (REINFORCE) estimator would work but has high variance. Instead, sample u from a fixed noise source and push it through a deterministic differentiable map:

$$u = \mu_{\phi}(s) + \sigma_{\phi}(s) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad a = \tanh(u).$$

The `tanh` squash is needed because actions must live in $[-1, 1]^3$ (the `delta_x`, `delta_y`, `delta_z` of the EE action space), and an unbounded Gaussian would produce actions the environment clips, which silently breaks the density.

Squashing changes the density. For an invertible map $a = f(u)$, the change-of-variables formula gives

$$\log \pi(a | s) = \log \mu(u | s) - \log \left| \det \frac{\partial f}{\partial u} \right|.$$

Since `tanh` is applied elementwise, the Jacobian is diagonal with entries $1 - \tanh^2(u_i)$, so

$$\log \pi(a | s) = \log \mu(u | s) - \sum_{i=1}^{|A|} \log(1 - \tanh^2(u_i)). \quad (2.4)$$

Computed naively, $1 - \tanh^2(u)$ underflows to 0 for $|u| \gtrsim 9$ in float32 and the log becomes $-\infty$. Rewrite it:

$$1 - \tanh^2 u = \operatorname{sech}^2 u = \left(\frac{2}{e^u + e^{-u}} \right)^2 = \frac{4e^{-2u}}{(1 + e^{-2u})^2},$$

$$\log(1 - \tanh^2 u) = \log 4 - 2u - 2 \log(1 + e^{-2u}) = 2 \left[\log 2 - u - \operatorname{softplus}(-2u) \right].$$

That final expression is exactly what `torch.distributions.TanhTransform.log_abs_det_jacobian` implements, and LeRobot gets it for free by wrapping a `MultivariateNormal` in a `TransformedDistribution` (`TanhMultivariateNormalDiag`, `modeling_gaussian_actor.py:633`). The base distribution's `log_prob` already sums over action dimensions, and `TransformedDistribution` subtracts the summed log-det, so `log_probs` comes back as one scalar per batch element. Sampling uses `rsample()` (`modeling_gaussian_actor.py:471`), which is the reparameterised path; `sample()` would detach the gradient and the actor would never learn.

One parametrisation note: `Policy.forward` computes `std = torch.exp(log_std)` and then clamps in `std` space to `[std_min, std_max] = [1e-5, 5]` from `policy_kwargs`, rather than the more common clamp of `log_std` to `[-20, 2]`. The comments in the file say this matches the JAX reference.

Automatic temperature tuning

A fixed α is a poor choice because the scale of the reward changes over training (early on, all Q-values are near zero, so any α is huge in relative terms). Pose the constrained problem instead:

$$\max_{\pi} \mathbb{E} \left[\sum_t r(s_t, a_t) \right] \quad \text{s.t.} \quad \mathbb{E}_{(s,a) \sim \rho_{\pi}} [-\log \pi(a | s)] \geq \bar{\mathcal{H}} \quad \forall t.$$

Form the Lagrangian with multiplier $\alpha \geq 0$ and take the dual. For a fixed policy, the dual objective in α is

$$J(\alpha) = \mathbb{E}_{a \sim \pi_\phi} \left[-\alpha (\log \pi_\phi(a | s) + \bar{\mathcal{H}}) \right]. \quad (2.5)$$

This is linear in α with slope $-(\log \pi + \bar{\mathcal{H}})$. Minimising it drives α up when the current entropy $-\log \pi$ falls below the target $\bar{\mathcal{H}}$ (the policy is collapsing, so buy more exploration) and down when it exceeds it. α is stored as `log_alpha` so it stays positive by construction (`sac_algorithm.py:118`).

2.6 The three losses, explicitly

With B the minibatch, N critics, and y from (2.3):

Critic loss (`_compute_loss_critic`, mean over batch per critic, then summed over critics):

$$L_Q(\theta) = \sum_{i=1}^N \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} \left(Q_{\theta_i}(s,a) - y \right)^2, \quad y \text{ computed under } \texttt{no_grad}.$$

Actor loss (`_compute_loss_actor`, using the *online* critics, not the targets):

$$L_\pi(\phi) = \frac{1}{|B|} \sum_{s \in B} \left[\alpha \log \pi_\phi(a_\phi(s, \varepsilon) | s) - \min_i Q_{\theta_i}(s, a_\phi(s, \varepsilon)) \right].$$

This is precisely the KL in (2.2) with $\log Z(s)$ dropped and the sign flipped.

Temperature loss (`_compute_loss_temperature`, with $\log \pi$ detached):

$$L_\alpha = \frac{1}{|B|} \sum_{s \in B} \left[-e^{\log \alpha} (\log \pi_\phi(a | s) + \bar{\mathcal{H}}) \right].$$

2.7 Symbol to config key

Every key below lives under `algorithm:` in `train_config.json`, defined in `src/lerobot/rl/algorithms/sac/configuration_sac.py`.

Symbol	Config key	Class default	Sim value used
γ	<code>discount</code>	0.99	0.97
τ	<code>critic_target_update_weight</code>	0.005	0.005
α_0	<code>temperature_init</code>	1.0	0.01
$\bar{\mathcal{H}}$	<code>target_entropy</code>	None	None
N	<code>num_critics</code>	2	2
$-\alpha \log \pi(a' s')$ in (2.3)	<code>use_backup_entropy</code>	True	<code>true</code>
critic steps per actor step	<code>utd_ratio</code>	1	2
actor update cadence	<code>policy_update_freq</code>	1	1
gradient clip	<code>grad_clip_norm</code>	40.0	10.0

When `target_entropy` is `None`, `_init_temperature` sets $\bar{\mathcal{H}} = -\frac{1}{2}(|\mathcal{A}_{\text{cont}}| + \mathbb{1}[\text{discrete head}])$. For our sim task that is $-(3+1)/2 = -2.0$. This is *not* Haarnoja's $-|\mathcal{A}|$ heuristic; it is half of it, so LeRobot targets a lower entropy and therefore a more deterministic policy than the reference. Worth knowing before you conclude your policy is under-exploring.

Two subtleties. `utd_ratio: 2` means one extra critic-only gradient step per call to `update()`, each on a *fresh* minibatch, before the final step that also updates actor and temperature. And `policy_update_freq` does not behave like TD3's delayed policy update: `sac_algorithm.py:245` runs `freq` actor updates every `freq` steps, so the actor:critic update ratio stays 1:1 and all `freq` repeats reuse the same minibatch `fb`.

Finally, the naming trap. `algorithm.type` is `"sac"`, but `policy.type` is `"gaussian_actor"`, not `"sac"`. LeRobot v0.6.1 split the algorithm (critics, temperature, Bellman update) from the policy (actor plus observation encoder), and the policy is registered as `gaussian_actor` in `configuration_gaussian_actor.py:84`. Passing `--policy.type=sac` will fail to resolve.

2.8 Two undocumented divergences from the reference implementation

These are not mentioned in the LeRobot docs, and both are cheap, high-value ablations for a thesis chapter.

SiLU versus tanh in the MLPs. `CriticHead`, `MLP`, and the actor network all default to `activations=nn.SiLU()` (`sac_algorithm.py:602`, `modeling_gaussian_actor.py:327`). The JAX reference HIL-SERL implementation uses `tanh` in these MLPs. Bounded-activation critics behave measurably differently under high UTD ratios, because tanh saturation limits how fast a critic head can chase a moving target. LayerNorm is applied in both cases, which mitigates but does not eliminate the difference.

use_backup_entropy=True versus False. LeRobot defaults to including $-\alpha \log \pi(a'|s')$ in the backup; the reference HIL-SERL configuration disables it. With `temperature_init: 0.01` the term is small at first, but α is learned, so if the temperature loss drives α up the entropy bonus starts contaminating the value estimate, and reported Q-values stop being comparable to discounted returns. If you are plotting Q against actual episode return in WandB, this is the flag that explains the offset.

Chapter 3: Learning from Prior Data (RLPD) and the Replay Buffer

3.1 The offline-to-online problem

You have demonstrations. In sim, `lilkm/pick_cube_franka_panda_30` gives you 458 transitions. On the SO-101 you will record something comparable, twenty or thirty teleoperated episodes. You also have a robot that can generate fresh data online, but slowly: at `fps: 10`, one hundred thousand environment steps is roughly 2.8 hours of continuous motion, before resets, before anything breaks. The question is how to use both.

The obstacle is distribution shift, and it is worth stating precisely because it drives every design decision below. The offline data was generated by some behaviour policy β (you, holding a gamepad), inducing a state-action occupancy $d^\beta(s, a)$. The critic is trained by minimising the Bellman residual against the backup

$$(\mathcal{T}^\pi Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P, a' \sim \pi(\cdot|s')} [Q(s', a') - \alpha \log \pi(a'|s')].$$

Notice where the expectation is taken: over a' drawn from the *current* policy, not from the data. If π puts mass on actions that β never took, the backup queries Q at inputs outside the support of the training distribution. A neural network asked to extrapolate returns whatever its weights happen to imply out there, and the errors are not symmetric: the `min` over the critic ensemble suppresses overestimates only among the critics that saw the region, and the actor is explicitly optimising $\max_a Q(s, a)$, so it *seeks out* whichever region has the largest extrapolation error. That error then propagates backwards through the backup into states the data does cover. This is the standard divergence spiral, and it is the reason offline RL needed a decade of conservatism penalties.

3.2 Why pretrain-then-finetune fails

The obvious recipe is: train a critic offline, then fine-tune online. It fails in two distinct ways.

If you pretrain with plain SAC on the offline buffer, you get exactly the spiral above, because nothing has bounded the extrapolation. You start online with a value function that is confidently wrong.

If you pretrain with a conservative offline algorithm (CQL, IQL), you avoid the spiral but you have now optimised a *different* objective, one deliberately biased toward pessimism on unseen actions. Switching to an unregularised online objective at step zero produces a documented performance dip: the values must first unlearn their pessimism, and while that happens the policy is worse than the behaviour policy you cloned from.

There is a third failure that matters specifically for HIL-SERL. If online fine-tuning samples only from a growing online buffer, the demonstrations are diluted geometrically. After ten thousand online steps, your 458 demonstration transitions are 4% of the buffer and contribute 4% of the gradient. In a sparse-reward task where the demonstrations are the *only* source of successful outcomes, that is catastrophic forgetting of the only signal you have.

3.3 RLPD's three ingredients

RLPD (Reinforcement Learning with Prior Data) resolves this by refusing to have an offline phase at all. It is an online algorithm from step zero that happens to carry a second buffer. Three ingredients make that work.

Symmetric sampling. Every minibatch is drawn half from the offline buffer and half from the online buffer, so the effective training distribution is the fixed mixture

$$d^{\text{mix}} = \frac{1}{2} d^{\mathcal{D}_{\text{off}}} + \frac{1}{2} d^{\mathcal{D}_{\text{on}}}.$$

Two things follow. The demonstrations never dilute: their share of the gradient is pinned at 50% forever, independent of how much online data accumulates. And the Bellman residual is minimised on a state distribution that covers both the expert's states (where the reward is) and the agent's states (where the policy

actually goes), which is precisely the coverage condition the backup needs. There is no objective mismatch to unlearn, because there was only ever one objective.

LayerNorm in the critic. This is the cheap trick that replaces conservatism penalties. An MLP with unbounded activations can emit arbitrarily large outputs for inputs far from the data manifold, and the magnitude grows with distance. Inserting `LayerNorm` after each hidden linear layer forces the features to zero mean and unit variance per sample, so the representation handed to the next layer cannot scale with how far off-manifold the input is. Extrapolation still happens, but it is bounded rather than divergent, which is enough to keep the actor from chasing a value that does not exist. In LeRobot this lives in the shared `MLP` builder, `src/lerobot/policies/gaussian_actor/modeling_gaussian_actor.py`, which appends `nn.LayerNorm(out_dim)` after every hidden `nn.Linear`; `CriticHead` in `src/lerobot/rl/algorithms/sac/sac_algorithm.py` wraps that MLP with a scalar output layer.

Ensembles with high update-to-data ratio. RLPD uses ten critics and takes the minimum over a random subset of two. The minimum over a random subset is a crude lower confidence bound: where the ensemble agrees (in-distribution) the min is close to the mean, where it disagrees (off-distribution) the min is sharply pessimistic. Pessimism is therefore applied *only where it is warranted*, without a global penalty term. LeRobot exposes both knobs, `num_critics` and `num_subsample_critics` in `configuration_sac.py`, with the subsampling implemented at `sac_algorithm.py:298` via `torch.randperm`.

3.4 Update-to-data ratio and what it costs

The update-to-data ratio G is the number of gradient steps taken per environment transition collected. RLPD uses $G = 20$. More gradient steps per sample is more sample efficiency, which is exactly what you want when samples cost robot time. The ensemble and LayerNorm are what make $G = 20$ survivable rather than a fast route to overfitting the critic to a small buffer.

In LeRobot, the loop is explicit (`sac_algorithm.py:201`):

```
for _ in range(self.config.utd_ratio - 1):
    batch = next(batch_iterator)
    fb = self._prepare_forward_batch(batch, include_complementary_info=True)
    loss_critic = self._compute_loss_critic(fb)
    ...
self._update_target_networks()
```

Each of those iterations pulls a *fresh* batch and does a critic-only update; the actor and temperature are updated once, on the final iteration. So `utd_ratio` scales critic work only.

The cost is not the critic MLPs (`latent_dim: 64`, and `CriticEnsemble` shares a single encoder across all heads, so extra heads are nearly free). The cost is the encoder forward pass: each batch pushes `batch_size` images per camera for both `state` and `next_state` through ResNet-10, plus the DrQ random-shift augmentation. On the 4090 with `utd_ratio: 2` the learner ran at roughly 6 to 7 Hz. The actor runs at 10 Hz. In the asynchronous actor/learner architecture, dropping the learner rate does not slow the robot down, it makes the robot's policy staler. At $G = 20$ the learner would sit well under 1 Hz and the actor would be acting on weights hundreds of environment steps out of date.

3.5 Why the de-tuned defaults are defensible

Knob	RLPD	LeRobot HIL-SERL
<code>num_critics</code>	10	2
<code>utd_ratio</code>	20	1 to 2
<code>discount</code>	0.99	0.97

Knob	RLPD	LeRobot HIL-SERL
temperature_init	1.0	0.01

`utd_ratio` is bounded by the staleness argument above: on real hardware, sample efficiency bought by more gradient steps is paid for in policy lag.

`num_critics: 2` is a smaller loss than it looks. Pessimism from ensemble disagreement matters most when the online data is generated by an unguided exploratory policy. Under HIL-SERL, a human is steering the arm back toward the demonstrated manifold whenever it drifts, so the off-distribution queries that the ensemble was defending against are rarer by construction.

`discount: 0.97` sets an effective horizon of $1/(1 - \gamma) \approx 33$ steps, or 3.3 seconds at 10 Hz, versus 100 steps (10 s) at 0.99. It also caps the value scale, since $|Q| \leq r_{\max}/(1 - \gamma)$, from 100 down to 33, shrinking the magnitude of the bootstrap error that LayerNorm has to contain. Pick-and-place episodes are a few seconds long with a terminal reward, so 33 steps is enough to see the goal from the start of the episode.

`temperature_init: 0.01` is the safety argument. α weights the entropy bonus in both the actor loss and (with `use_backup_entropy: true`) the backup. RLPD initialises at 1.0 because it must explore from nothing. Here, exploration is supplied by the human, and supplied in the right place, near the bottleneck where the policy actually fails. A high-entropy Gaussian over end-effector deltas at 10 Hz is, physically, an arm that shakes; that is wear on STS3215 gearboxes for exploration you are getting for free from the gamepad. α is still learned against `target_entropy`, so 0.01 is an initialisation and not a cap.

The summary claim, and the one to defend in a viva: *human intervention and high UTD buy the same thing, sample efficiency in the region that matters, and the human version is cheaper in wall-clock and safer for the hardware.*

3.6 The ReplayBuffer

`src/lerobot/rl/buffer.py` is a flat, pre-allocated ring buffer, not a deque of transition objects. Storage is allocated lazily on the first `add()`, with shapes inferred from that transition (`_initialize_storage`, lines 134 to 187). `add()` and `sample()` are guarded by `self._lock` because the learner samples on one thread while a second thread drains the gRPC transition queue. Sampling applies DrQ augmentation (`random_shift`, `pad=4`) to state and next-state images concatenated into a single call, which is why `use_drq` defaults to `True`.

Mixing happens one level up, in `OnlineOfflineMixer` (`src/lerobot/rl/data_sources/data_mixer.py`):

```
n_online = max(1, int(batch_size * self.online_ratio))
n_offline = batch_size - n_online
online_batch = self.online_buffer.sample(n_online)
offline_batch = self.offline_buffer.sample(n_offline)
return concatenate_batch_transitions(online_batch, offline_batch)
```

With `mixer: "online_offline"` and `online_ratio: 0.5` this is literally RLPD symmetric sampling. Set `online_ratio: 1.0` (or omit `dataset`) and the offline buffer disappears, giving you plain SAC.

`optimize_memory` is the one flag you cannot set from the config: `learner.py` hardcodes `optimize_memory=True` for both the online buffer (line 805) and the offline buffer (line 861). When it is on, `self.next_states = self.states` is a bare reference and `sample()` reads the successor as `states[(idx + 1) % capacity]`. This halves image storage. The correctness caveat is at episode boundaries: for a genuine terminal the factor $(1 - \text{done})$ in

```
td_target = rewards + (1 - done) * self.config.discount * min_q
```

zeroes the bootstrap, so the wrong `next_state` is multiplied by zero and does no harm. For a time-limit truncation it would matter, and `truncateds` is stored by the buffer but is not consulted anywhere in `_compute_`

`loss_critic`.

3.7 The float32 image trap, with arithmetic

`_initialize_storage` allocates with no `dtype`:

```
self.states = {
    key: torch.empty((self.capacity, *shape), device=self.storage_device)
    for key, shape in state_shapes.items()
}
```

`torch.empty` with no `dtype` gives `torch.get_default_dtype()`, that is **float32, 4 bytes per element**, for image tensors. The memory for image storage in one buffer is

$$M = N \cdot K \cdot 3 \cdot H \cdot W \cdot b \cdot (2 - 1[\text{optimize_memory}]),$$

with N the capacity, K the number of cameras, $b = 4$ bytes. With `optimize_memory=True` the trailing factor is 1.

At the default `online_buffer_capacity: 100000` and `offline_buffer_capacity: 100000`:

Resolution	Cameras	Per buffer	Both buffers
128x128	1	19.66 GB (18.31 GiB)	39.32 GB (36.62 GiB)
128x128	2	39.32 GB (36.62 GiB)	78.64 GB (73.24 GiB)
64x64	1	4.92 GB (4.58 GiB)	9.83 GB (9.16 GiB)
64x64	2	9.83 GB (9.16 GiB)	19.66 GB (18.31 GiB)

Double every figure if `optimize_memory` were `False`. The sim setup (two cameras, 3x128x128) therefore asks for 73 GiB of image storage on a machine with 61 GB of RAM and a 24 GB card.

Two facts explain why it nonetheless runs. First, `storage_device: cpu: torch.empty` on CPU reserves virtual address space, and under Linux overcommit the physical pages are faulted in only as transitions are actually written. With 458 offline transitions and a few thousand online, resident memory stays small even though the tensors nominally span 73 GiB. Second, on CUDA the caching allocator calls `cudaMalloc` immediately, so the same configuration with `storage_device: cuda` dies at allocation time, not gradually. **Do not set `storage_device: cuda`.**

Note also that `from_lerobot_dataset` allocates the *full* capacity, not the dataset length: a 458-transition demo set with `offline_buffer_capacity: 100000` still reserves 100000 slots. Set `offline_buffer_capacity` to the dataset size and `online_buffer_capacity` to what you will actually collect (50000 is still ~1.4 hours at 10 Hz), and crop aggressively with `src/lerobot/rl/crop_dataset_roi.py`, which reduces H and W quadratically. Storing images as `uint8` and converting at sample time would cut all of the above by a factor of four, but that is a change to `buffer.py`, not a configuration flag.

Chapter 4: Human-in-the-Loop and the Reward Problem

Part A: The Intervention Mechanism

What HIL-SERL adds on top of RLPD

RLPD gave us an off-policy learner that can consume a fixed demonstration set and online rollouts in the same gradient step. HIL-SERL adds exactly one thing to that recipe: during online collection, a human can seize control of the robot at any timestep, and the transitions produced while they hold control go into the same replay buffers as everything else.

That is the whole idea, and its cheapness is the point. There is no separate imitation loss, no auxiliary behaviour-cloning term, no DAgger-style relabelling pass. The human's actions enter as data, not as supervision.

The reason this is legal comes straight from the Bellman backup. The critic target used in `sac_algorithm.py` is

$$y = r + \gamma(1 - d) \left[\min_{i \in \{1,2\}} Q_{\theta_i}(s', a') - \alpha \log \pi_{\phi}(a' | s') \right], \quad a' \sim \pi_{\phi}(\cdot | s')$$

and the critic loss is $\mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} [(Q_{\theta_i}(s, a) - y)^2]$. Note where the actions come from. The action a' at which the target network is evaluated is drawn from the *current* policy. The action a that is fed into Q_{θ_i} is read from the buffer, and nothing in the derivation cares which policy produced it. Off-policy TD learning is indifferent to the behaviour policy. It is not indifferent to the action *space*.

How a takeover becomes a transition

Concretely, on each actor step (`src/lerobot/rl/actor.py`), the policy proposes an action, the processor pipeline runs, and `InterventionActionProcessorStep` in `src/lerobot/processor/hil_processor.py` decides what actually gets recorded:

```
is_intervention = info.get(TeleopEvents.IS_INTERVENTION, False)
if is_intervention and teleop_action is not None:
    if isinstance(teleop_action, dict):
        action_list = [
            teleop_action.get("delta_x", 0.0),
            teleop_action.get("delta_y", 0.0),
            teleop_action.get("delta_z", 0.0),
        ]
        if self.use_gripper:
            action_list.append(teleop_action.get(GRIPPER_KEY, 1.0))
        teleop_action_tensor = torch.tensor(action_list, dtype=action.dtype, device=action.device)
        new_transition[TransitionKey.ACTION] = teleop_action_tensor
```

The policy's proposal is discarded and the human's action is written into `TransitionKey.ACTION`. Downstream, the environment executes that action, and the resulting $(s, a_{\text{human}}, r, s')$ is serialised to the learner like any other transition. The `is_intervention` flag rides along in `complementary_info`, and `learner.py` uses it for one extra thing:

```
replay_buffer.add(**transition)
# Add to offline buffer if it's an intervention
if dataset_repo_id is not None and transition.get("complementary_info", {}).get(
    TeleopEvents.IS_INTERVENTION.value
):
    offline_replay_buffer.add(**transition)
```

So an intervention transition is added twice: once to the online buffer and once to the demonstration buffer. With `mixer: "online_offline"` and `online_ratio: 0.5`, that means every corrective action you make is

treated as a fresh demonstration and is sampled at the elevated demo rate for the rest of training. Interventions are expensive per unit of human time and correspondingly up-weighted.

The `is_intervention` flag itself comes from the teleoperator through a deliberately minimal protocol (`hil_processor.py`):

```
@runtime_checkable
class HasTeleopEvents(Protocol):
    def get_teleop_events(self) -> dict[str, Any]: ...
```

returning `is_intervention`, `terminate_episode`, `success`, and `rerecord_episode`. `_check_teleop_with_events` raises a `TypeError` naming `GamepadTeleop` and `KeyboardEndEffectorTeleop` if the device does not implement it.

Why the recorded action must live in the policy's action space

The critic is trained on buffer actions and the actor is improved by ascending the critic at its own reparameterised samples:

$$\mathcal{L}_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}, \epsilon \sim \mathcal{N}} \left[\alpha \log \pi_\phi(a_\phi(s, \epsilon) \mid s) - \min_i Q_{\theta_i}(s, a_\phi(s, \epsilon)) \right]$$

The actor's only learning signal is $\nabla_a Q(s, a)$ evaluated where the actor samples. If the buffer contains actions from a different parameterisation, the critic is being fit on one set of (s, a) pairs while the actor queries it on a disjoint set. Q becomes an accurate function on a manifold the actor never touches and an unconstrained extrapolation everywhere the actor actually lives. Worse, the human's contribution to shaping Q near the actor's own samples is zero, so the interventions do nothing at all except waste your time.

The SO-101 leader arm is the concrete case, and it fails in both a loud and a silent way. The loud failure: `S0Leader` in `src/lerobot/teleoperators/so_leader/so_leader.py` has no `get_teleop_events()`, so `_check_teleop_with_events` raises at `hil_processor.py:89` before the actor loop starts. PR #3086, which adds leader-arm support, is open and unmerged against v0.6.1.

The silent failure is the one to understand, because it is what you would get if you patched around the protocol check. `S0Leader.get_action()` returns

```
action = self.bus.sync_read("Present_Position", num_retry=self.config.num_read_retries)
action = {f"{motor}.pos": val for motor, val in action.items()}
```

which is a dict keyed `shoulder_pan.pos`, `elbow_flex.pos`, and so on: absolute joint positions in degrees. `InterventionActionProcessorStep` looks up `"delta_x"`, `"delta_y"`, `"delta_z"`, finds none of them, and falls through to the defaults. Every intervention step would record the action $(0, 0, 0)$ while the follower arm is visibly moving under your hand. The critic then learns $Q(s, 0)$ is high in exactly those states where you rescued the robot, the actor happily converges on emitting zeros, and nothing in the logs looks wrong. The `TypeError` is a feature: it prevents this. On v0.6.1, use the gamepad (or `keyboard_ee`), which emits `{"delta_x": 0, "delta_y": 1, "delta_z": 2, "gripper": 3}` matching the 3-continuous-plus-1-discrete action space exactly.

Why long interventions hurt

The paper is explicit that interventions should be short corrective nudges, not takeovers that carry the episode to success. The value-overestimation argument runs as follows. Suppose you grab control at state s_t and drive a 40-step trajectory ending in $r = 1$. TD learning propagates that return backwards: $Q(s_t, a_t^{\text{human}})$ rises towards γ^{40} . But the target at s_t is bootstrapped through $\min_i Q_{\theta_i}(s_{t+1}, a')$ with $a' \sim \pi_\phi$, the policy's action, and the policy cannot produce the remaining 39 steps. The critic is therefore asserting that a high return is attainable from s_{t+1} under π_ϕ , which is false. That optimistic value then leaks backwards into the states the policy does reach on its own, and the actor climbs a gradient towards a region whose value is an artefact of your hand. The

behavioural symptom is a policy that drives confidently into the neighbourhood where you usually take over and then stalls, waiting to be rescued. Short interventions avoid this because the policy is forced to finish, so the reward is credited to a state-action sequence it can actually generate. The double-insertion into the offline buffer compounds the problem: long takeovers flood the demo buffer with near-identical human data and quietly turn 50/50 mixing into mostly-human batches.

The ablation, and the metric that matters

The published ablation on this is stark: no demonstrations and no interventions gives 0% success, demonstrations alone give 49%, demonstrations plus interventions give 100%. The zero is not surprising under a sparse reward, since a policy that never stumbles into success has no signal to learn from. The interesting gap is 49 versus 100. Demonstrations tell the critic what a good return looks like, but they were collected under the human's state distribution, and once the policy drifts off it there is no data at the point of failure. Interventions place labelled data exactly at the boundary of the policy's competence, which is the active-learning argument for DAGger, except that here the currency is value rather than action labels.

This is why *intervention rate*, not success rate, is the honest progress metric. `actor.py` computes it per episode and logs it to WandB:

```
intervention_rate = episode_intervention_steps / episode_total_steps
```

alongside a boolean `Episode intervention`. Success rate is confounded, because a run can sit at 100% success indefinitely while you do all the work. A healthy curve starts high (roughly 0.5 to 0.8 over the first tens of episodes), decays roughly monotonically over a few hundred episodes towards below 0.1, and the `Episode intervention` boolean starts going false for whole episodes. Success rate rises as intervention rate falls; that anti-correlation is what learning looks like. A flat intervention rate means the actor is not receiving improved weights (check `policy_parameters_push_frequency`, which the official example sets to 50 seconds while the class default is 4 and the docs recommend 1 to 2) or the reward is misspecified. A rising rate after a period of decline usually means critic divergence.

Part B: The Reward Problem

In simulation, reward is free

The sim reward in `gym_hil/envs/panda_pick_gym_env.py:_compute_reward`, sparse branch, is three lines:

```
block_pos = self._data.sensor("block_pos").data
lift = block_pos[2] - self._z_init
return float(lift > 0.1)
```

The dense branch is $0.3 \exp(-20d) + 0.7 \text{clip}\left(\frac{z - z_{\text{init}}}{z_{\text{succ}} - z_{\text{init}}}, 0, 1\right)$. Both read privileged state. MuJoCo owns the ground truth, so the reward is exact, free, and impossible for the policy to fake: there is no input to `_compute_reward` other than the true block position.

On real hardware, `self._data.sensor("block_pos")` does not exist. You have two USB cameras and joint encoders. Nothing tells you the cube is 10 cm above the table.

Option 1: a human presses a button

`get_teleop_events()` returns `success`, and `InterventionActionProcessorStep` acts on it directly:

```
new_transition[TransitionKey.DONE] = bool(terminate_episode) or (self.terminate_on_success and success)
new_transition[TransitionKey.REWARD] = float(success)
```

This is exact and requires no extra infrastructure, which makes it the right way to start. Its costs are that a human must supervise every episode forever, and that human reaction time enters the label: at `fps: 10` one

frame is 100 ms, so a late press mislabels one to three frames.

Option 2: a learned success classifier

`RewardClassifierProcessorStep` (`hil_processor.py`) loads `lerobot.rewards.classifier.modeling_classifier.Classifier` and is configured through `RewardClassifierConfig` in `src/lerobot/envs/configs.py` with `pretrained_path`, `success_threshold: 0.5`, `success_reward: 1.0`.

`Architecture` (`configuration_classifier.py`): a `lerobot/resnet10` encoder, `num_cameras: 2`, `SpatialLearnedEmbeddings` pooling with `image_embedding_pooling_dim: 8`, projection to `latent_dim: 256`, concatenation across cameras, an MLP head with `hidden_dim: 256` and `dropout_rate: 0.1`, and, because `num_classes: 2`, a single output unit trained with `binary_cross_entropy_with_logits` and squashed by a sigmoid. It is deliberately the same ResNet-10 backbone the policy uses, so the visual features are the ones you are already committed to.

It trains on labelled frames from recorded episodes: frames in the success configuration are positives, everything else negative.

`terminate_on_success` must be false during data collection and true during RL. During collection (`ResetConfig.terminate_on_success = false`), the episode does not end at the first success, so the operator can hold the object in the success configuration and vary its pose, yielding many diverse positives instead of one frame per episode. During RL it must be true, and the reason is arithmetic. With `discount: 0.97`, a policy that reaches success and parks there collects $\sum_t \gamma^t = 1/(1 - 0.97) \approx 33$, versus 1 for terminating. That 33x distortion dominates the value function and rewards loitering over completing the task.

Class imbalance. One success frame in a 200-step episode is 0.5% positives, and a classifier that always predicts "fail" scores 99.5% accuracy. Aim for something in the 1:3 to 1:5 positive:negative range by holding success poses during collection and subsampling the long uninteresting approach phase, and report balanced accuracy or precision at the operating threshold rather than raw accuracy.

Split by episode, never by frame. At 10 fps, consecutive frames are near-duplicates. A random frame-level split places frame t in train and frame $t+1$ in validation, so validation accuracy measures memorisation, not generalisation. This temporal leakage routinely produces 99% validation numbers on a classifier that fails on the next session's lighting. Hold out whole episodes; expect the honest number to be considerably lower, and treat that as information rather than as a bug.

Adversarial states. Once the classifier is the reward, the policy is a dedicated optimiser searching the classifier's input space for high output. Every region where the classifier is wrong is free return, and RL is unusually good at finding those regions. The characteristic failure is a policy that positions the gripper so the wrist camera sees something that looks like a held block, or occludes the view at the moment the classifier is most confident, without ever having grasped anything.

The cheap defences are structural, and both are already in the code. First, use the hard threshold: `success_reward` is emitted only when $p > \text{success_threshold}$, never the probability itself. A continuous reward of p would hand the policy a dense gradient to climb straight into the adversarial region; a step function gives it nothing until it crosses, so exploitation requires stumbling onto the far side rather than following a slope there. Second, `terminate_on_success: true` caps the payoff of any exploit that is found at a single $+1$, instead of the ≈ 33 that a parked policy would farm. Two cameras help, since an exploit has to fool both views simultaneously. Beyond that, the practical loop is DAGger on the reward model: watch for episodes that terminate without the task being done, add those frames as negatives, retrain. One last mundane trap: the classifier sees whatever the pipeline hands it, so the crop and resize parameters (`ImageCropResizeProcessorStep`, set with `crop_dataset_roi.py`) must be identical between classifier training and RL, or you are evaluating it off-distribution from the first step.

Chapter 5: Kinematics and the End-Effector Action Space

Everything the policy in this project emits is a small Cartesian displacement. Nothing in `sac_algorithm.py` knows what a servo is. The translation from "move 4 mm along $+z$ " to six Feetech goal positions happens entirely inside a processor chain built on a URDF and a numerical IK solver. This chapter derives that machinery, then argues why HIL-SERL puts the learning problem in end-effector space at all, and finally shows exactly how much of the general theory LeRobot v0.6.1 actually implements (less than you might assume: three translational degrees of freedom and nothing else).

5.1 Forward kinematics as a product of homogeneous transforms

A rigid-body pose is an element of $SE(3)$, represented as a 4×4 homogeneous transform

$$T = \begin{bmatrix} R & p \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad R \in SO(3), p \in \mathbb{R}^3,$$

acting on homogeneous points $\tilde{x} = (x, y, z, 1)^\top$. The point of the 4×4 form is that rotation and translation compose under ordinary matrix multiplication, so a serial chain of n joints has

$$T_n^0(q) = \prod_{i=1}^n T_i^{i-1}(q_i).$$

Forward kinematics is nothing more than evaluating this product. The two common parameterisations of the factors differ only in bookkeeping. Denavit-Hartenberg pins each link frame to a canonical convention so that a link needs exactly four numbers, giving $T_i^{i-1} = \text{Rot}_z(\theta_i) \text{Trans}_z(d_i) \text{Trans}_x(a_i) \text{Rot}_x(\alpha_i)$. URDF, which is what we use, drops the convention and stores an arbitrary fixed offset per joint (`<origin xyz rpy>`) plus a unit axis $\hat{a}$ (`<axis>`):

$$T_i^{i-1}(q_i) = T_{\text{origin},i} \cdot \begin{bmatrix} e^{\hat{a}_i^\wedge q_i} & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix} \quad (\text{revolute}),$$

with the matrix exponential given by Rodrigues' formula $e^{\hat{a}^\wedge \theta} = I + \sin \theta \hat{a}^\wedge + (1 - \cos \theta)(\hat{a}^\wedge)^2$, where $\hat{a}^\wedge$ is the skew-symmetric matrix of $\hat{a}$. URDF is more verbose but strictly more expressive and machine-readable, which is why every modern solver ingests it.

The SO-101 follower exposes six motors, in this order (`src/lerobot/robots/so_follower/so_follower.py`): `shoulder_pan`, `shoulder_lift`, `elbow_flex`, `wrist_flex`, `wrist_roll`, `gripper`. Only the first five move the tip frame, so the arm is a **5-DoF** chain driving a 6-D pose. Hold that thought.

`src/lerobot/model/kinematics.py` wraps this in about forty lines:

```
self.robot = placo.RobotWrapper(urdf_path)
self.solver = placo.KinematicsSolver(self.robot)
self.solver.mask_fbase(True) # fix the base link
self.tip_frame = self.solver.add_frame_task(self.target_frame_name, np.eye(4))
```

`forward_kinematics` converts degrees to radians, calls `self.robot.set_joint(name, q)` for each joint, `update_kinematics()`, and returns `get_T_world_frame(target_frame_name)`, a raw 4×4 NumPy array. Two practical requirements follow. First, you need the URDF: for this arm, `so101_new_calib.urdf` from `TheRobotStudio/SO-ARM100` (`Simulation/SO101/`), with `target_frame_name` set to the tip link (`gripper_frame_link` is the default in `RobotKinematics.__init__`). Second, the solver assumes **degrees** on the wire (`np.deg2rad` on input, `np.rad2deg` on output), which matches the follower's default `use_degrees: bool = True`. Flip that flag and your kinematics silently becomes wrong rather than crashing.

5.2 The manipulator Jacobian

Differentiating the FK map $\mathbf{x} = \mathbf{f}(\mathbf{q})$ gives the Jacobian $\mathbf{J}(\mathbf{q}) = \partial \mathbf{f} / \partial \mathbf{q}$, which maps joint velocities to the end-effector **twist** $\mathbf{V} = (\mathbf{v}, \boldsymbol{\omega}) \in \mathbb{R}^6$:

$$\begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}, \quad \mathbf{J} \in \mathbb{R}^{6 \times n}.$$

For a revolute joint i with world-frame axis $\hat{\mathbf{z}}_i$ and origin $\mathbf{p}_i$, the geometric Jacobian column has the clean form

$$\mathbf{J}_i = \begin{bmatrix} \hat{\mathbf{z}}_i \times (\mathbf{p}_e - \mathbf{p}_i) \\ \hat{\mathbf{z}}_i \end{bmatrix},$$

the linear part being the moment arm from the joint to the tip. This single equation carries most of the intuition: a joint's influence on tip position scales with its distance from the tip, so wrist joints move the tip slowly and precisely while the shoulder moves it fast and coarsely. A fixed-magnitude perturbation in joint space therefore produces a tip displacement whose size and direction depend entirely on $\mathbf{q}$. We will use that against joint-space learning shortly.

Note also that the `gripper` joint contributes a **zero column** to $\mathbf{J}$ for the tip frame (opening the fingers does not move `gripper_frame_link`), yet `gym_manipulator.py` passes all six motor names as `joint_names` to `RobotKinematics`. The solver is handed a structurally rank-deficient problem by construction.

5.3 Inverse kinematics, damping, and singularities

IK asks for $\mathbf{q}$ such that $\mathbf{f}(\mathbf{q}) = \mathbf{x}^*$. There is no closed form for a general URDF chain, so we pose it as nonlinear least squares,

$$\mathbf{q}^* = \arg \min_{\mathbf{q}} \frac{1}{2} \|\mathbf{f}(\mathbf{q}) - \mathbf{x}^*\|_{\mathbf{W}}^2,$$

and iterate. Linearising around the current $\mathbf{q}$ with $\Delta \mathbf{x} = \mathbf{x}^* - \mathbf{f}(\mathbf{q})$ gives the Gauss-Newton step $\mathbf{J} \Delta \mathbf{q} \approx \Delta \mathbf{x}$, whose minimum-norm solution is $\Delta \mathbf{q} = \mathbf{J}^+ \Delta \mathbf{x}$. The pseudoinverse is the problem: near a singularity, where $\mathbf{J}$ loses rank, $\mathbf{J}^+$ diverges and the arm is commanded to lash out at enormous joint velocity for a tiny Cartesian request.

Levenberg-Marquardt fixes this by penalising the step itself:

$$\Delta \mathbf{q} = \arg \min_{\Delta \mathbf{q}} \|\mathbf{J} \Delta \mathbf{q} - \Delta \mathbf{x}\|^2 + \lambda^2 \|\Delta \mathbf{q}\|^2.$$

The normal equations give $(\mathbf{J}^\top \mathbf{J} + \lambda^2 \mathbf{I}) \Delta \mathbf{q} = \mathbf{J}^\top \Delta \mathbf{x}$, and the push-through identity converts this to the standard damped-least-squares form,

$$\Delta \mathbf{q} = \mathbf{J}^\top (\mathbf{J} \mathbf{J}^\top + \lambda^2 \mathbf{I})^{-1} \Delta \mathbf{x}$$

which is cheaper when $n > 6$. The SVD makes the effect transparent: with $\mathbf{J} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^\top$,

$$\Delta \mathbf{q} = \sum_i \frac{\sigma_i}{\sigma_i^2 + \lambda^2} \mathbf{v}_i (\mathbf{u}_i^\top \Delta \mathbf{x}).$$

Undamped, the gain is $1/\sigma_i$, unbounded as $\sigma_i \rightarrow 0$. Damped, the gain peaks at $1/(2\lambda)$ when $\sigma_i = \lambda$ and decays to zero as $\sigma_i \rightarrow 0$. Damping trades exact tracking for boundedness: near a singularity the solver gives up on the unreachable direction instead of exploding. Since λ caps the step, it is also an implicit velocity limit.

Placo implements this as a QP with **soft tasks**. `RobotKinematics.inverse_kinematics` configures the frame task with `("soft", position_weight, orientation_weight)`, defaulting to `position_weight=1.0, orientation_weight=0.01`, then calls `self.solver.solve(True)`. Those weights are the $\mathbf{W}$ in the objective above and they matter enormously on this arm: a 5-DoF chain generically **cannot** hit an arbitrary 6-D pose, so the residual is never zero and the weights decide which part of the request gets sacrificed. At a 100:1 ratio, position wins and

orientation is advisory. The RL path (`InverseKinematicsRLStep`) calls `inverse_kinematics` without passing a weight, so it inherits `0.01`.

5.4 Why HIL-SERL learns in end-effector delta space

The LeRobot documentation (`docs/source/hilserl.mdx`) makes the empirical claim bluntly: "learning in joint space for reinforcement learning in manipulation is often a harder problem: some tasks are nearly impossible to learn in joint space but become learnable when the action space is transformed to end-effector coordinates." Four mechanisms explain that.

Task-relevant geometry. Reward depends on the tip pose relative to objects, which is a function of $f(q)$, not q . In EE space the action coordinates are the same coordinates the reward is written in. In joint space the policy must learn f implicitly before it can even represent a straight-line reach.

A smoother optimisation landscape. The critic must learn $Q(s, a)$. If $a = \Delta q$, then the resulting tip motion is $J(q)\Delta q$, so the mapping from action to outcome is state-dependent and badly conditioned: identical actions produce wildly different displacements depending on configuration, and Gaussian exploration noise $\Delta q \sim \mathcal{N}(0, \sigma^2 I)$ becomes the anisotropic Cartesian ellipsoid $\mathcal{N}(0, \sigma^2 J J^\top)$, collapsing to nothing near singularities. In EE space the IK layer absorbs J^{-1} so exploration noise is isotropic and consistent in the space that matters.

Transferability. An EE-space policy is a statement about the task, not the arm. Change link lengths, or move to a different 6-DoF arm, and the policy is at least meaningful.

Human compatibility, which for HIL-SERL is decisive. The operator holding the Xbox pad pushes a stick and the tip moves that way. Interventions and demonstrations are recorded in the *same* action space the policy outputs, so corrections land in the replay buffer as directly usable on-policy-ish labels. If the human commanded joints and the policy commanded Cartesian deltas, the two data sources would not be commensurable.

5.5 What LeRobot actually implements: 3-DoF translation

The general theory above allows full 6-D control. LeRobot v0.6.1 does not use it. `MapTensorToDeltaActionDictStep` (`src/lerobot/processor/delta_action_processor.py`) slices the policy's output tensor into exactly this:

```
delta_action = {"delta_x": action[0].item(),
                "delta_y": action[1].item(),
                "delta_z": action[2].item()}
if self.use_gripper:
    delta_action["gripper"] = action[3].item()
```

with a literal `# TODO (maractingi): add rotation` above it. `MapDeltaActionToRobotActionStep` then hardcodes `target_wx = target_wy = target_wz = 0.0`, and `EEReferenceAndDelta` computes `desired[:3,:3] = ref[:3,:3] @ Rotation.from_rotvec([0,0,0]).as_matrix()`, i.e. the identity. **The commanded orientation is always "whatever the reference orientation currently is."** It is held, not controlled, and since the RL path runs with `use_latched_reference=False` and `use_ik_solution=True`, the reference is the FK of the *previous IK solution*, so orientation drifts wherever the soft-weighted solver leaves it.

So the learned action is $a \in \mathbb{R}^3 \times \{0, 1, 2\}$: three continuous translational deltas plus a discrete gripper command (`num_discrete_actions=3`, `close/stay/open`). The consequence for task selection is concrete. Choose tasks solvable with a roughly fixed wrist attitude: top-down pick and place, pushing, button pressing, vertical peg insertion, dropping objects into a bin. Avoid anything requiring reorientation: screwing a cap, pouring, flipping a part, or inserting at an angle the reset pose does not already provide. If your task needs the wrist to rotate, you must either fix that orientation mechanically in the reset pose or extend the action space yourself.

5.6 The processor chain

The canonical teleoperation chain (`examples/so100_to_so100_EE/teleoperate.py`) is `EEReferenceAndDelta` → `EEBoundsAndSafety` → `InverseKinematicsEEToJoints`, with `GripperVelocityToJoint` slotted in. The RL path in `src/lerobot/rl/gym_manipulator.py` (around lines 500-520) is the one that runs during training, and its exact order is:

```
MapTensorToDeltaActionDictStep(use_gripper=...) # tensor -> delta_x/y/z, gripper
MapDeltaActionToRobotActionStep()               # -> enabled, target_*, gripper_vel
EEReferenceAndDelta(kinematics=..., end_effector_step_sizes=...,
                    use_latched_reference=False, use_ik_solution=True)
EEBoundsAndSafety(end_effector_bounds=...)
GripperVelocityToJoint(clip_max=..., speed_factor=1.0, discrete_gripper=True)
InverseKinematicsRLStep(kinematics=..., motor_names=...,
                        initial_guess_current_joints=False)
```

`GripperVelocityToJoint` must precede the IK step, because the IK step pops `ee.gripper_pos` and writes `gripper.pos` directly, bypassing the solver's own (meaningless) gripper output. Each step is a small pure function on a transition dict, which is what makes the chain serialisable and inspectable.

Two behaviours in this chain are worth knowing before you debug a "frozen" robot. `MapDeltaActionToRobotActionStep` sets `enabled = ||delta|| > noise_threshold` with `noise_threshold = 1e-3`, so a policy emitting sub-millimetre deltas is treated as disabled and the chain re-sends the last command. And `EEBoundsAndSafety` defaults to `max_ee_step_m=0.05` with `raise_on_jump=True`, meaning an over-limit per-frame step raises `ValueError` and aborts the control loop rather than clamping.

5.7 Bounds and step sizes: the highest-leverage knob you have

`InverseKinematicsConfig` (`src/lerobot/envs/configs.py`) carries four fields: `urdf_path`, `target_frame_name`, `end_effector_bounds` (`{"min": [...], "max": [...]}`, metres), and `end_effector_step_sizes` (`{"x":..., "y":..., "z":...}`, metres per unit action). The documented SO-100 example is a box roughly 8 cm × 28 cm × 7 cm with 2 cm steps:

```
"end_effector_bounds": {"min": [0.16, -0.08, 0.03], "max": [0.24, 0.2, 0.1]},
"end_effector_step_sizes": {"x": 0.02, "y": 0.02, "z": 0.02}
```

`end_effector_step_sizes` converts the policy's `[-1, 1]` output into metres, so at `fps: 10` a value of 0.02 caps tip speed at 0.2 m/s. `end_effector_bounds` clips the absolute target inside `EEBoundsAndSafety` via `np.clip`. Derive both by running `lerobot-find-joint-limits` with the leader arm and moving the follower through exactly the region that solves the task.

Tight bounds are the single highest-leverage lever in this whole pipeline for two independent reasons. Safety is the obvious one: the arm physically cannot reach the table edge, the camera mount, or itself. The subtler one is sample efficiency. Online RL cost scales with the volume of state-action space that exploration must cover, and the reachable Cartesian volume is a direct multiplier on that. Shrinking the box from 30 cm to 8 cm on one axis removes most of the trajectories the agent would otherwise waste episodes on, and it does so without any reward shaping, which means without introducing a shaped-reward bias you would then have to defend. It is the cheapest possible prior: pure geometry, zero hyperparameters, exactly encoding "the task lives here."

The cost to be aware of is that clipping is not a soft penalty. A policy pushing outward against a bound sees its action produce no state change, which looks in the logs like a stalled agent rather than a saturated one. If your critic's Q-values flatten and the arm sits still at a workspace corner, check the bounds before you touch the learning rate.

Chapter 6: The Simulation Practicum

Everything in this chapter is something you already ran. The point of writing it down is that the sim run is your only clean reference: it is the one configuration where every component worked and you know it worked. When the hardware run misbehaves, the diagnostic question is always "what is different from the sim run?", and you can only ask that if you know precisely what the sim run was.

6.1 MuJoCo and what gym_hil adds on top

MuJoCo is a rigid-body physics engine: you hand it an MJCF/XML scene (bodies, joints, geoms, actuators, sensors), it integrates the constrained equations of motion, and it exposes the resulting state through two structs, `model` (static, compiled from XML) and `data` (dynamic, the current state). It is not a robot framework and it has no notion of tasks, rewards, or episodes.

`gym_hil` supplies exactly that missing layer for a Franka Panda with a Robotiq 2F-85 gripper. The scene lives in `gym_hil/assets/` (`scene.xml`, `panda.xml`, `franka_emika_panda/`, `robotiq_2f85/`). `gym_hil/mujoco_gym_env.py` defines `MujocoGymEnv` and `FrankaGymEnv`, which own the physics stepping, the operational-space controller (`gym_hil/controllers/opspace.py`), the Cartesian workspace bounds, and offscreen rendering. `gym_hil/envs/panda_pick_gym_env.py` defines the task. `gym_hil/wrappers/` supplies the human-in-the-loop layer: `EEActionWrapper` (map normalised deltas to end-effector motion), `GripperPenaltyWrapper`, `InputsControlWrapper` (gamepad or keyboard takeover), `PassiveViewerWrapper` (the on-screen MuJoCo viewer), `ResetDelayWrapper`.

Two timescales matter: `physics_dt = 0.002` is the integrator step; `control_dt = 0.1` is the agent step. One `env.step()` runs 50 physics substeps, which is exactly the `fps: 10` in every HIL-SERL config. That 10 Hz is the number you carry over to hardware, and it is why control-loop latency on the real arm is a first-class concern: you have 100 ms per decision, total.

6.2 The registered environments, and the trap in `PandaPickCubeBase-v0`

`gym_hil/__init__.py` registers, for the pick-cube task:

- `gym_hil/PandaPickCubeBase-v0`, entry point `gym_hil.envs:PandaPickCubeGymEnv` (the raw env, no wrappers)
- `gym_hil/PandaPickCubeViewer-v0`, the base wrapped in `PassiveViewerWrapper`
- `gym_hil/PandaPickCube-v0`, `...Gamepad-v0`, `...Keyboard-v0`, all with entry point `gym_hil.wrappers.factory:make_env` and different kwargs

All have `max_episode_steps=100`, which at 10 Hz is a 10-second episode.

`Gamepad-v0` and `Keyboard-v0` both set `use_viewer: True` in their registration kwargs, so `PassiveViewerWrapper` is applied and a MuJoCo window opens. That window is not optional cosmetics, and it becomes relevant in the segfault below.

Now the trap. LeRobot builds the env in `src/lerobot/rl/gym_manipulator.py:323`:

```
env = gym.make(
    f"gym_hil/{cfg.task}",
    image_obs=True,
    render_mode="human",
    use_gripper=use_gripper,
    gripper_penalty=gripper_penalty,
)
```

`use_gripper` and `gripper_penalty` are parameters of `make_env / wrap_env` in `gym_hil/wrappers/factory.py`, not of `PandaPickCubeGymEnv.__init__`, whose signature is `(seed, control_dt, physics_dt, render_spec, render_mode, image_obs, reward_type, random_block_position)`. So setting `env.task` to `PandaPickCubeBase-v0` raises a `TypeError` on an unexpected keyword argument before anything else happens. LeRobot always passes these kwargs; only the factory-routed IDs accept them. Use `PandaPickCubeGamepad-v0` (which is what `configs/sim/train_config.json` sets) or `PandaPickCubeKeyboard-v0`.

Note also that `image_obs=True` is hard-coded, not configurable. That has consequences.

6.3 The observation and the reward

The reward, verbatim from `_compute_reward`:

```
block_pos = self._data.sensor("block_pos").data

if self.reward_type == "dense":
    tcp_pos = self._data.sensor("2f85/pinch_pos").data
    dist = np.linalg.norm(block_pos - tcp_pos)
    r_close = np.exp(-20 * dist)
    r_lift = (block_pos[2] - self._z_init) / (self._z_success - self._z_init)
    r_lift = np.clip(r_lift, 0.0, 1.0)
    return 0.3 * r_close + 0.7 * r_lift
else:
    lift = block_pos[2] - self._z_init
    return float(lift > 0.1)
```

The default is `reward_type="sparse"`, so $r_t = \mathbb{1}[z_{\text{block}} - z_{\text{init}} > 0.1]$ with `self._z_init` cached at reset and `_z_success = _z_init + 0.1`. It costs nothing, it is exact, and it never lies. Remember that when you get to reward classifiers.

The observation, from `_compute_observation`:

```
if self.image_obs:
    front_view, wrist_view = self.render()
    observation = {
        "pixels": {"front": front_view, "wrist": wrist_view},
        "agent_pos": robot_state,
    }
else:
    observation = {
        "agent_pos": robot_state,
        "environment_state": block_pos,
    }
```

This is the single most important structural fact in the sim task. `block_pos` is read from the same MuJoCo sensor in both `_compute_reward` and `_compute_observation`, but it only enters the observation in the state-only branch. Since LeRobot forces `image_obs=True`, the policy receives `pixels` and `agent_pos` and never sees the cube's coordinates. It must infer object position from 128x128 RGB. The reward function, by contrast, has privileged access to ground truth. That asymmetry (privileged reward, non-privileged observation) is exactly the structure HIL-SERL assumes, and on hardware you have to manufacture the privileged side yourself with a reward classifier.

Through LeRobot's processors this becomes `observation.images.front` and `observation.images.wrist`, both $3 \times 128 \times 128$, plus `observation.state` of dimension 18: 7 joint positions, 7 joint velocities, 1 gripper, 3 end-effector xyz . The action is 3 continuous deltas ($\Delta x, \Delta y, \Delta z$) plus a discrete gripper head with `num_discrete_actions: 3`, encoded close = 0, stay = 1, open = 2 (`gym_hil/wrappers/hil_wrappers.py:194-199`), assembled in `InterventionActionProcessorStep` in `src/lerobot/processor/hil_processor.py` from `teleop_action.get("delta_x")` and friends.

6.4 Actor and learner: two processes, one gRPC socket

HIL-SERL runs as two OS processes that never share memory.

The **actor** (`src/lerobot/rl/actor.py`) owns the environment, the gamepad, and a CPU/GPU copy of the policy used only for inference. It steps at `fps`, blends human and policy actions, and streams transitions out. The **learner** (`src/lerobot/rl/learner.py`) owns the replay buffers, the critics, the temperature, the optimisers, and the gradient steps. It never touches the environment.

They talk over gRPC on `127.0.0.1:50051` (`policy.actor_learner_config.learner_host` / `learner_port`). The service is defined in `src/lerobot/transport/services.proto`:

```
service LearnerService {
  rpc StreamParameters(empty) returns (stream Parameters);
  rpc SendTransitions(stream Transition) returns (empty);
  rpc SendInteractions(stream InteractionMessage) returns (empty);
  rpc Ready(empty) returns (empty);
}
```

The actor opens three long-lived streams: one server-streaming (parameters flowing down) and two client-streaming (transitions and interaction statistics flowing up). Everything is `bytes`, chunked with a `TransferState` enum because parameter blobs exceed gRPC's 4 MB default message limit.

The parameter push is time-driven, not step-driven. In `learner.py:420`:

```
if time.time() - last_time_policy_pushed > policy_parameters_push_frequency:
    push_actor_policy_to_queue(parameters_queue=parameters_queue, algorithm=algorithm)
    last_time_policy_pushed = time.time()
```

`policy_parameters_push_frequency` is therefore **seconds between pushes**, despite reading like a frequency. The class default in `configuration_gaussian_actor.py:59` is 4; the official example config ships 50; the docs recommend 1 to 2. At 50 s and 10 Hz the actor collects roughly 500 steps on stale weights before it ever sees an update, which quietly destroys the on-policy-ness the whole design depends on. On loopback the serialisation cost is negligible, so your run used 2, and the learner log confirms `'policy_parameters_push_frequency': 2`.

6.5 Launching the run

Two terminals, learner first (it binds the port), both with the venv activated because `.venv/bin/python` bypasses the activation script's path setup.

```
# terminal 1
source .venv/bin/activate
python -m lerobot.rl.learner \
  --config_path configs/sim/train_config.json \
  --output_dir=runs/wandb_learner \
  --job_name=simsmoke-panda-pickcube \
  --wandb.project=hilserl-so101 \
  --policy.actor_learner_config.policy_parameters_push_frequency=2
```

```
# terminal 2
source .venv/bin/activate
python -m lerobot.rl.actor \
  --config_path configs/sim/train_config.json \
  --output_dir=runs/wandb_actor \
  --job_name=simsmoke-panda-pickcube \
  --wandb.project=hilserl-so101 \
  --policy.actor_learner_config.policy_parameters_push_frequency=2
```

Both processes parse the **same** config file; draccus dotted overrides let you diverge only where needed (here, `output_dir`). Two facts about that config worth restating: `policy.type` must be `"gaussian_actor"` (not `"sac"`, which was the pre-PR-#3075 name), while `algorithm.type` stays `"sac"`; and `policy.storage_device` must be `"cpu"`, because `ReplayBuffer._initialize_storage` in `src/lerobot/rl/buffer.py` allocates with `torch.empty(...)` and no `dtype`, giving float32 image tensors. Two cameras at $3 \times 128 \times 128$ float32 across a 100000-capacity buffer is far past 24 GB.

A third fact deserves its own paragraph, because the shipped config and the source disagree. `configs/sim/train_config.json` sets `"use_torch_compile": true`, while `sac_algorithm.py:93-94` carries the comment `# TODO(Khalil): Investigate and fix torch.compile followed by # NOTE: torch.compile is disabled, policy does not converge when enabled.` The flag is still honoured at line 95, so the example config enables a code path the authors have annotated as non-convergent. Pass `--algorithm.use_torch_compile=false` until that comment disappears from the source.

6.6 What to watch in WandB, in order

1. **Intervention rate** (logged from `actor.py:411`, computed as `episode_intervention_steps / episode_total_steps`). This is the primary HIL-SERL health signal. It should fall. If it stays flat, the policy is not absorbing your corrections and nothing else matters.
2. **Episodic reward** (`actor.py:408`). With the sparse reward this is effectively the per-episode success indicator. Noisy, but the trend is the task.
3. **loss_critic** (`sac_algorithm.py:230`), plus `loss_actor` and `loss_temperature`. Watch for divergence, not for a pretty curve. A critic loss that climbs monotonically while intervention rate is falling usually means the target is chasing a moving buffer distribution.
4. **Optimization frequency loop [Hz]** (`learner.py:441`). Yours ran at roughly 6 to 7 Hz on the 4090, against an actor at 10 Hz. Learner slower than actor is normal and fine here; learner an order of magnitude slower is not.
5. **GPU memory**, via `nvidia-smi` alongside. Flat after the first hundred steps is what you want. A slow climb means something is accumulating.

6.7 Why this worked, and why none of it transfers

The sim task converged easily for four reasons, and every one of them is an artefact of simulation:

- **The reward is free and exact.** One float comparison on a MuJoCo sensor; no false positives, no labelling. On hardware you train a `Classifier` (`src/lerobot/rewards/classifier/modeling_classifier.py`) on your own annotated frames, and every misfire is either a phantom reward the policy will farm or a missed success it can never credit.
- **458 transitions of demonstration were preloaded.** `lilkm/pick_cube_franka_panda_30` (30 episodes) seeds the offline buffer, and with `mixer: "online_offline"` and `online_ratio: 0.5` half of every batch is expert data from step one. That is a large fraction of the learning signal, handed to you.
- **Resets are perfect.** `reset()` calls `mujoco.mj_resetData`, drops the arm at `_PANDA_HOME`, and places the block at exactly **(0.5, 0.0)** unless `random_block_position` is set. Your SO-101 has no `mj_resetData`. `ResetConfig.fixed_reset_joint_positions` drives the arm back, but the object stays wherever the last episode left it, which means a human resets the scene between every episode.
- **No latency, no drift, no wear.** Sim observations are exact and instantaneous. Real cameras drop frames, USB bandwidth is finite, Feetech STS3215 servos heat up and shift, and calibration drifts over a session.

The sim run proves your stack is wired correctly: gRPC, buffers, encoders, gamepad, WandB. It proves nothing about your task.

6.8 Troubleshooting two failures you will hit again

Missing-gamepad segfault. In `gym_hil/wrappers/intervention_utils.py`, `GamepadController.__init__` sets `self.controller_config = None`; `start()` checks `pygame.joystick.get_count() == 0` and, if so, prints a warning and `return`s, leaving `controller_config` as `None`. The subsequent `update()` does `self.controller_config.get("buttons", {})` and raises `AttributeError` on a background polling thread. Because `use_viewer=True`, that thread is racing the MuJoCo passive viewer thread, and the interpreter tears down into `glfw.terminate()` from the wrong thread. The result is a segfault, not a traceback, so the actual cause never prints. Plug the Xbox Series X controller in **before** starting the actor, and confirm SDL sees it (`Initialized gamepad: Xbox Series X Controller` on stdout) before you trust anything downstream.

Actor-restart hang. `src/lerobot/rl/learner_service.py:42` sets `MAX_WORKERS = 3` with the comment "Stream parameters, send transitions and interactions", and `learner.py:522` builds the server with `ThreadPoolExecutor(max_workers=MAX_WORKERS)`. Three threads, three long-lived RPCs, one actor. `StreamParameters` never checks `context.is_active()`, so when an actor dies without a clean shutdown its half-open streams pin all three threads indefinitely. The new actor connects at the TCP level, sends `Ready`, and waits forever for a worker that will never free. The symptom is a silent hang with no error on either side. The rule is unconditional: **when the actor dies, kill and restart the learner too, every time**. Since the learner is where the replay buffer lives, that also means checkpoint frequently on hardware, because a restart loses the online buffer.

Chapter 7: The Real Robot Pipeline

Everything up to this point has been simulation, where a reset is a function call and a camera never moves. On hardware, every one of those free operations becomes a physical act performed by you. This chapter is the execution plan: seven stages, each with a goal, the exact commands, what goes in, what comes out, a go/no-go gate you must pass before spending time on the next stage, and an honest time estimate. The gates matter more than the commands. Almost every failed HIL-SERL attempt on a low-cost arm is a stage that was passed over and then silently invalidated everything downstream.

Stage A: Hardware bring-up and device stability

Goal. Make `/dev` names deterministic across reboots and replugs, and make two USB cameras coexist on one machine at 10 Hz without dropping frames.

Why this is stage A and not stage E. Two things you will configure later are expressed as opaque integers tied to a physical device: the serial port each arm answers on, and the pixel rectangle `crop_params_dict` applied to each camera stream. If `/dev/video0` and `/dev/video2` swap identity after a reboot, the front crop is applied to the wrist image. Nothing raises. The dataset you spent three hours recording is now garbage in a way you will not notice until training fails to move. The same applies to ports: the follower connected on the leader's port receives the leader's calibration from `follower_arm` versus `leader_arm` and moves to the wrong joint offsets.

Commands.

```
lsusb                                # find VID:PID for each device
udevadm info -a -n /dev/ttyACM0 | grep -E 'idVendor|idProduct|serial|KERNELS'
udevadm info -a -n /dev/video0 | grep -E 'idVendor|idProduct|ATTR{index\}'
```

Write `/etc/udev/rules.d/99-hilserl.rules`:

```
# Arms: Feetech USB-serial adapters
SUBSYSTEM=="tty", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", KERNELS=="1-3.1",
SYMLINK+="so101_follower"
SUBSYSTEM=="tty", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", KERNELS=="1-3.2",
SYMLINK+="so101_leader"

# Cameras: index==0 selects the real capture node
SUBSYSTEM=="video4linux", ATTRS{idVendor}=="046d", ATTRS{idProduct}=="0825", ATTR{index}=="0", KERNELS=="1-4", SYMLINK+="cam_front"
SUBSYSTEM=="video4linux", ATTRS{idVendor}=="046d", ATTRS{idProduct}=="0825", ATTR{index}=="0", KERNELS=="2-1", SYMLINK+="cam_wrist"
```

```
sudo udevadm control --reload-rules && sudo udevadm trigger
sudo usermod -aG dialout $USER # then log out and back in
id -nG | tr ' ' '\n' | grep dialout
```

The `ATTR{index}=="0"` clause. A UVC device does not register one `/dev/video*` node. Since Linux 4.16 it registers at least two: the video capture node and a metadata node, and some webcams register more. All of them share the same `idVendor` / `idProduct`. A rule keyed only on VID:PID therefore matches every node the device exposes, and the symlink lands on whichever one udev processed last. Half your boots the camera works, half of them `cv2.VideoCapture` opens a node that never returns a frame. `ATTR{index}` is the V4L2 node ordinal within the device, and index 0 is the capture node.

If both Feetech adapters are the same CH340 chip with no unique `serial` attribute (common), you cannot discriminate on VID:PID and must key on `KERNELS`, which is the physical USB port path. That is fine, but it means the rule now says "whatever is plugged into this socket", so label both cables and both sockets physically.

USB bandwidth. USB 2.0 high speed gives 480 Mbit/s per host controller, and the UVC driver reserves isochronous bandwidth at open time based on the alternate setting's `dwMaxPayloadTransferSize`, not on the bytes you actually consume. Many webcams advertise a worst-case payload, so opening the second camera on the same controller fails with `ENOSPC` ("No space left on device") even though two MJPG streams would fit comfortably. Check the topology with `lsusb -t` and move one camera to a port hanging off a different root hub. Forcing MJPG also matters: uncompressed YUYV at 640x480x30 is about 147 Mbit/s per camera before overhead, and two of those genuinely do saturate one controller.

Forcing MJPG and V4L2 together. `OpenCVCameraConfig` (`src/lerobot/cameras/opencv/configuration_opencv.py`) defaults `backend` to `Cv2Backends.ANY` (value 0). `camera_opencv.py:166` passes that straight into `cv2.VideoCapture(self.index_or_path, self.backend)`, and with `ANY` a full opencv-python build may resolve a `/dev/video*` path through FFMPEG or GStreamer. Those backends do not implement `CAP_PROP_FOURCC`, so your `fourcc="MJPG"` is accepted and ignored. Set both:

```
{ "type": "opencv", "index_or_path": "/dev/cam_front", "fps": 30,
  "width": 640, "height": 480, "fourcc": "MJPG", "backend": 200 }
```

`200` is `Cv2Backends.V4L2`; draccus also accepts the string `"V4L2"` (verified). With `V4L2` selected, `_validate_fourcc` reads the `FOURCC` back from the device and raises if it did not stick, which is exactly the failure you want to be loud.

Gate. Reboot twice. Both `/dev/so101_*` and both `/dev/cam_*` resolve to the same physical devices each time. Run `lerobot-teleoperate` with both cameras enabled for five minutes with no `ENOSPC` and no frame-timeout warnings. **Time: 2 to 4 hours.**

Stage B: URDF and workspace bounds

Goal. Get a working IK chain and a deliberately small end-effector box.

```
git clone https://github.com/TheRobotStudio/S0-ARM100.git
# URDF: S0-ARM100/Simulation/S0101/so101_new_calib.urdf
```

Resolve the frame name first. The codebase contains two different defaults for the same thing: `RobotKinematics.__init__` in `src/lerobot/model/kinematics.py` defaults `target_frame_name="gripper_frame_link"`, while `FindJointLimitsConfig` in `src/lerobot/scripts/lerobot_find_joint_limits.py` defaults to `"gripper"`. Do not trust either. Enumerate:

```
import placio
r = placio.RobotWrapper("/abs/path/S0-ARM100/Simulation/S0101/so101_new_calib.urdf")
print(r.frame_names())
```

Use the identical string in `--target_frame_name` here and in `env.processor.inverse_kinematics.target_frame_name` later.

```
lerobot-find-joint-limits \
  --robot.type=so101_follower --robot.port=/dev/so101_follower --robot.id=follower_arm \
  --teleop.type=so101_leader --teleop.port=/dev/so101_leader --teleop.id=leader_arm \
  --urdf_path=/abs/path/S0-ARM100/Simulation/S0101/so101_new_calib.urdf \
  --target_frame_name=<resolved> \
  --warmup_time_s=5 --teleop_time_s=60 --control_loop_fps=30
```

`--urdf_path` is a required field with no default (it is a bare `urdf_path: str` in the dataclass), and the tutorial prose omits it. Without it draccus aborts before touching the robot.

This is also the one place the leader arm is genuinely useful. `lerobot-find-joint-limits` only calls `teleop.get_action()` and forwards joint positions, which `S0Leader` provides. It cannot drive HIL-SERL interventions,

because `hil_processor.py:89` expects `delta_x / delta_y / delta_z` keys and `get_teleop_events()`, neither of which `SOLeader` has on v0.6.1.

Choosing the box. The script prints the full swept volume. Do not paste that in. Take the smallest axis-aligned box containing your intended demonstration trajectories, add roughly 2 cm of margin, and use that:

```
"inverse_kinematics": {
  "urdf_path": "...", "target_frame_name": "...",
  "end_effector_bounds": { "min": [0.16, -0.08, 0.03], "max": [0.24, 0.20, 0.10] },
  "end_effector_step_sizes": { "x": 0.02, "y": 0.02, "z": 0.02 }
}
```

The bounds are a hard `np.clip` on the commanded target (`robot_kinematic_processor.py:233`), so they are simultaneously a safety cage and a reduction of the state space the policy must cover. With a few thousand real transitions, a 12 cm box is a tractable problem and a 40 cm box is not.

Gate. Teleoperate for two minutes deliberately trying to escape: the arm clips at the box faces, never reaches a joint hard stop, never contacts the table. **Time: 1 to 2 hours.**

Stage C: Demonstrations

Goal. 15 to 30 successful episodes in end-effector action space at `fps: 10`, recorded through exactly the pipeline that will run online.

```
python -m lerobot.rl.gym_manipulator --config_path configs/real/env_sol1_pick.json
```

with `"mode": "record"`, `dataset.repo_id`, `dataset.root`, `dataset.num_episodes_to_record: 30`, `processor.control_mode: "gamepad"`, `processor.gripper.use_gripper: true`.

The reset pose must be exact. `processor.reset.fixed_reset_joint_positions` is the vector that `reset_follower_position` in `gym_manipulator.py` interpolates to over 50 steps at the start of every episode, online and offline. If your demonstrations begin from a pose that differs from it, then the offline data and the online rollouts have disjoint state distributions at $t = 0$, and the critic is asked to bootstrap $Q(s_0, a)$ at states no demonstration ever visited. The fix is procedural, not numerical: teleoperate to the intended start pose, read the joint vector, paste it into the config, and from then on record only with the reset applied, so the first recorded frame is the reset pose by construction.

Reset randomisation. Move the object by about 1 cm between episodes, in a pattern you write down. Zero randomisation gives you a policy that has memorised one trajectory and is unfalsifiable as a result. Large randomisation means 30 demonstrations no longer cover the distribution and the demonstrations stop helping.

The reset arithmetic, stated plainly. At `fps: 10` with `control_time_s: 20.0` and `reset_time_s: 5.0`, a full-length episode occupies 25 s of wall clock and 200 environment steps. Thirty demonstrations is 30 resets. A reward-classifier dataset (Stage E) is another 20 to 40. A single online run of 20,000 steps is on the order of 100 to 200 episodes depending on how often `terminate_on_success` fires early, and you should expect three to five runs before anything works. Across the whole experiment that is **150 to 750 physical resets**, each one you placing an object on a mark. At 10 s each, 750 resets is over two hours of nothing but resetting, spread across days of supervision. Design the task fixture so a reset is one motion, not three.

Gate. Set `"mode": "replay"` and `dataset.replay_episode: k` for two random k ; the arm reproduces the demonstration without hitting the box or stalling. Confirm the dataset has reward 1 only on terminal frames. **Time: 2 to 3 hours.**

Stage D: Crop the ROI and freeze the visual world

Goal. Remove nuisance pixels before the encoder ever sees them.

```
python -m lerobot.rl.crop_dataset_roi \
  --repo-id lucas/sol01_pick_cube --root data/sol01_pick_cube
```

Draw a rectangle per camera, press `c`. Output: a new dataset at `<root>_cropped_resized` and `meta/crop_params.json`. The call site hardcodes `resize_size=(128, 128)`, matching the encoder input shape.

Why cropping is not cosmetic. You are fitting a vision-conditioned critic on the order of 10^4 transitions with a frozen ResNet-10. Nothing in the objective prefers a causal feature over a spurious one. If the room lights dim as the afternoon progresses, and successes happen to cluster later in a session, the critic can read time-of-day off the background and get a lower Bellman residual for it. Cropping deletes the nuisance variable instead of asking a frozen encoder to become invariant to it, which it cannot do, since `freeze_vision_encoder: true` means the representation never adapts at all.

Three disciplines follow from the same argument. Fix the lighting: artificial only, blinds shut, and disable auto-exposure and auto-white-balance with `v4l2-ctl` so global image statistics do not drift between the demonstration session and the online run. Mount both cameras rigidly, to the table or an independent frame, and verify the arm at full extension does not enter the camera body's volume. A gooseneck that shifts 2 mm when the table is bumped silently invalidates every crop rectangle. Finally, copy `crop_params.json` verbatim into `env.processor.image_preprocessing.crop_params_dict` with `"resize_size": [128, 128]`, so the online environment applies the byte-identical transform the offline data went through.

Gate. Inspect ten frames spanning the whole session; gripper and object inside the crop in every one, at both the start pose and the success pose. **Time: 30 to 60 minutes, plus mounting.**

Stage E: Reward

Run 1: you are the reward function. In `hil_processor.py`, the human path is literal: `REWARD = float(success)` and `DONE = terminate_episode or (terminate_on_success and success)`. Pressing the success button writes a 1 into the buffer. This is the recommended first round: it removes an entire failure mode (a miscalibrated classifier) from your first debugging session, and it produces the labelled dataset the classifier needs, for free.

Run 2: the classifier. Collect a dedicated dataset with `processor.reset.terminate_on_success: false` so episodes continue past success. With `true`, positives are roughly one frame in two hundred and the classifier converges to predicting zero.

```
lerobot-train --config_path configs/real/reward_classifier.json
```

The config key trap. The model block is `reward_model`, not `policy`. `src/lerobot/scripts/lerobot_train.py:307` branches on `cfg.is_reward_model_training` and only then calls `make_reward_model`. Put your classifier under `policy` and you will train something else entirely on the classifier dataset, with no error. The block needs `"type": "reward_classifier"`, `model_name`, `num_cameras: 2`, `num_classes: 2`, and `input_features` naming both `3x128x128` image keys.

Deploy under `env.processor.reward_classifier: pretrained_path`, `success_reward: 1.0`, and `success_threshold: 0.7` rather than the 0.5 default. The asymmetry is deliberate. A false negative costs you one wasted episode. A false positive terminates the episode *and* writes reward 1 for a non-success state, which is a direct injection of error into the Bellman target $y = r + \gamma(1 - d) \min_i Q_{\bar{\theta}_i}(s', a')$, and the $d = 1$ also removes the bootstrap that would otherwise correct it. Then flip `terminate_on_success` back to `true`.

Gate. Run `gym_manipulator` with the classifier active and no policy for 20 episodes. Require under 2% false positives on non-success frames. **Time: 3 to 5 hours.**

Stage F: Training

Learner first (it opens the gRPC server), actor second, same config file:

```
python -m lerobot.rl.learner --config_path configs/real/train_sol01_pick.json
python -m lerobot.rl.actor   --config_path configs/real/train_sol01_pick.json
```

Mandatory overrides against the official sim example.

`policy.actor_learner_config.policy_parameters_push_frequency: 2`. The example JSON ships 50, and the unit is **seconds**, not steps. At `fps: 10` that means the actor executes 500 steps of a policy the learner has already replaced, and your interventions are attributed to a network that no longer exists. The class default is 4; the documentation recommends 1 to 2.

Buffer capacities down, hard. `ReplayBuffer._initialize_storage` (`src/lerobot/rl/buffer.py:147`) allocates with `torch.empty((capacity, *shape), device=self.storage_device)` and no `dtype`, so images are stored as float32 even though they arrive as uint8. One transition holds two `3x128x128` float32 images, which is $2 \times 3 \times 128 \times 128 \times 4 = 393,216$ bytes, about 0.375 MiB. The learner does at least pass `optimize_memory=True` (`learner.py:805, 824, 861`), so next-states are not duplicated. Even so, `online_buffer_capacity: 100000` plus `offline_buffer_capacity: 100000` is $200,000 \times 0.375$ MiB, roughly 75 GiB. That exceeds 61 GB of RAM and is absurd against 24 GB of VRAM. Set `online_buffer_capacity: 20000` (about 7.5 GiB) and `offline_buffer_capacity: 8000` (about 3 GiB); your demonstration set is only 3,000 to 6,000 transitions anyway. Keep `storage_device: "cpu"` so the 4090 is left entirely to the encoder, the critics, and the actor forward passes.

Starting hyperparameters (from the official `gym_hil` example, carried over unchanged unless noted):

Key	Value	Reason
<code>policy.type</code>	<code>gaussian_actor</code>	not "sac" ; that is the algorithm
<code>algorithm.type</code>	<code>sac</code>	
<code>algorithm.discount</code>	0.97	horizon $1/(1-\gamma) \approx 33$ steps = 3.3 s at 10 Hz
<code>algorithm.temperature_init</code>	0.01	low initial entropy weight; demos dominate early
<code>algorithm.utd_ratio</code>	2	gradient steps per env step
<code>algorithm.num_critics</code>	2	clipped double-Q
<code>algorithm.grad_clip_norm</code>	10.0	class default is 40.0
<code>policy.latent_dim</code>	64	
<code>policy.vision_encoder_name</code>	<code>lerobot/resnet10</code>	
<code>policy.freeze_vision_encoder</code>	true	
<code>algorithm.use_backup_entropy</code>	true	entropy term inside the target
<code>policy.online_step_before_learning</code>	100	
<code>mixer / online_ratio</code>	<code>online_offline</code> / 0.5	half of every batch from demos

Intervention discipline. Take over the moment the policy is about to fail, apply a short correction, and hand back immediately. Do not drive to success. Persistent long interventions that end in success bias the value function upward in exactly the region where the policy has no experience, which is the destabilisation the HIL-SERL paper warns about explicitly.

Healthy versus unhealthy. Healthy: intervention rate trending down over the first few thousand online steps, episode length shortening, critic loss noisy but bounded, learned temperature settling, learner holding roughly 6 to 7 Hz optimisation frequency (measured on this 4090 in sim), actor holding 10 Hz. Unhealthy: Q-value magnitudes growing without bound, intervention rate flat or rising, actor step frequency below `fps` (camera stalls or learner starvation), reward spikes that do not correspond to anything you saw happen. **Gate:** 1,000 online steps with no crash and a measurable move in intervention rate. **Time: 1 to 2 days per serious run, 3 to 5 runs.**

Stage G: Evaluation

Write the protocol down before you run it. Fix N (30 is a reasonable floor), fix the initial-state distribution as a set of marked positions drawn from the same 1 cm jitter used in training, and decide in advance who calls success and by what criterion. Add a held-out set at 3 cm offset to measure the generalisation cliff deliberately rather than discovering it in the viva. Report per-trial binary outcomes with a Wilson interval, not a bare mean: 24 of 30 is a 95% interval of roughly [0.63, 0.91], and quoting "80%" alone overstates what 30 trials can support.

Do not build on `src/lerobot/rl/eval_policy.py`. Line 58 reads `env = make_robot_env(env_cfg)`, but `make_robot_env` (`gym_manipulator.py:304`) is annotated `-> tuple[gym.Env, Any]` and returns `(env, teleop_device)`. The first `env.reset()` therefore raises on a tuple. Write your own short loop against `make_robot_env` plus `make_processors`, the way `gym_manipulator.main` does. You want that control anyway, because the trial protocol is the experiment. **Time: half a day per policy.**

Expectations

HIL-SERL was developed on stiff industrial arms with reliable closed-loop end-effector servoing. The SO-101 is a 3D-printed arm on hobby serial-bus servos with position-only control, meaningful backlash, and no force feedback, driven here at 10 Hz through delta-EE actions resolved by placo IK. The action noise floor is simply higher, and every stage above exists to buy back some of that margin. Budget weeks, not days, and expect the first two runs to be diagnostics rather than results.

One limitation should be stated rather than engineered away. The workspace box is tight and the reset randomisation is about 1 cm, so the resulting policy is competent over a narrow distribution and will degrade sharply outside it. That is not a bug in your setup: it is what a few thousand real transitions can cover. Heavy augmentation to disguise it would trade an honest, measurable scope claim for an unmeasurable one. Put the narrow-workspace result in the thesis as the baseline's actual scope, and if you want to widen it, widen it as the contribution.

Chapter 8: Hyperparameters, Failure Modes and Experimental Practice

This chapter is the reference you consult when a run misbehaves at 23:00 and you need to decide whether the problem is a number, a wire, or the algorithm. Part A fixes the numbers. Part B is the register of things that actually break. Part C is about running this as science rather than as a demo.

Part A: Hyperparameter reference

Two sources of truth matter here, and they disagree. The **class default** is what the dataclass gives you if you say nothing (`src/lerobot/rl/algorithms/sac/configuration_sac.py`, `src/lerobot/policies/gaussian_actor/configuration_gaussian_actor.py`, `src/lerobot/envs/configs.py`). The **shipped example** is what the official HIL-SERL config actually sets. Where they differ, the example is almost always right and the default is a generic SAC default that was never tuned for this setting.

Symbol	Config key	Class default	Shipped example	Recommended start	What it controls, and how it fails
α_0	<code>algorithm.temperature_init</code>	1.0	0.01	0.01	Initial entropy weight in $J = \mathbb{E}[Q - \alpha \log \pi]$. At 1.0 the policy is pinned to near-uniform exploration; on a real arm that is a 10 Hz random-walk EE jitter that destroys your demos' value and can hit joint limits. Too small and the policy collapses to a deterministic mode early, so intervention data is the only exploration you get.
$\tilde{H}$	<code>algorithm.target_entropy</code>	None $\Rightarrow -\dim(A)/2$	null	null	Sets the setpoint of the dual loss $J(\alpha) = \mathbb{E}[-\alpha(\log \pi + \tilde{H})]$ (<code>sac_algorithm.py:419</code>). With 3 continuous plus 1 discrete dimension the default is -2.0 (<code>sac_algorithm.py:125</code>). Making it less negative forces more noise and more collisions; more negative starves exploration and α decays to zero. Change this only after you have looked at the logged <code>temperature</code> .
UTD	<code>algorithm.utd_ratio</code>	1	2	2	Gradient steps per sampled batch group. Raising it buys sample efficiency, which is the scarce resource on hardware, but it lowers the learner's wall-clock optimization frequency (measured ~6-7 Hz on the 4090 at UTD 2) and therefore increases policy staleness at the actor. High UTD with <code>num_critics=2</code> and no subsampling overfits the critic.
γ	<code>algorithm.discount</code>	0.99	0.97	0.97	Effective horizon $1/(1-\gamma)$: 33 steps at 0.97, which at <code>fps: 10</code> is 3.3 s, about one pick. 0.99 gives 10 s, longer than the task, so credit is smeared across resets and the critic learns a near-constant value.

Symbol	Config key	Class default	Shipped example	Recommended start	What it controls, and how it fails
N	<code>algorithm.num_critics</code>	2	2	2	Ensemble size for the clipped double-Q target. Going to 1 gives runaway overestimation and a policy that drives into the table. Going above 2 costs VRAM and learner throughput for little gain at this scale.
c	<code>algorithm.grad_clip_norm</code>	40.0	10.0	10.0	Applied to actor, critic, discrete critic and log α separately. At 40.0 a single bad batch (a long human intervention with a large TD error) can move the critic far enough that recovery takes thousands of steps. Too small and learning stalls; watch the logged <code>grad_norms</code> .
d_z	<code>policy.latent_dim</code>	256	64	64	Width of the per-modality encoder output that is concatenated before the MLP trunks. 256 with two cameras plus state gives a 768-dim trunk input and slows the learner; 64 is the tuned value for 128×128 ResNet10 features.
T_{push}	<code>policy.actor_learner_config.policy_parameters_push_frequency</code>	4	50	1 to 2	Seconds between learner-to-actor parameter pushes. The shipped 50 is a trap: for 50 s the actor executes a frozen policy, so on-policy-ness collapses and you will conclude the algorithm does not learn. Set it to 1 or 2 unless gRPC serialization is measurably saturating.
-	<code>policy.storage_device</code>	"cpu"	"cpu"	"cpu"	Where replay tensors live. Set to "cuda" and you OOM (see B2). Keep on CPU, accept the per-batch host-to-device copy.
$ \mathcal{D} $	<code>policy.online_buffer_capacity</code> , <code>policy.offline_buffer_capacity</code>	100000 each	100000 each	online 25000, offline = size of your demo set	Pre-allocated ring buffers. 100000 is fine in sim (458 offline transitions) but on hardware with two 128×128 RGB cameras it reserves ~75 GiB of host RAM, more than your 61 GB. 25000 online is ~40 min at 10 Hz.
-	<code>env.fps</code>	30 (base EnvConfig)	10	10	Control and logging rate. Every HIL-SERL config uses 10. Raising it shortens the effective horizon at fixed γ , multiplies buffer memory per minute, and pushes the USB cameras past their bandwidth budget.
-	<code>env.processor.reset.control_time_s</code>	20.0	15.0	15 to 20	Episode time limit before truncation. Too short and the task is unsolvable so every episode is a zero-reward truncation; too long and each failure costs you a minute of a finite-length session.

Symbol	Config key	Class default	Shipped example	Recommended start	What it controls, and how it fails
-	<code>env. processor. inverse_ kinematics. end_effector_ step_sizes</code>	None	not used in the sim example	start ~ 0.01 m per axis, then tune	Metres of Cartesian delta per unit action, converted to joints by placo IK. Too large and one action saturates the IK, overshoots and slams the arm; too small and the reachable set per episode is tiny so the agent never sees reward. This is a derived starting point, not a verified value: measure the actual displacement per step before trusting it.
-	<code>env. processor. gripper. gripper_ penalty</code>	0.0	-0.02	-0.02	Per-step cost on gripper actuation, shaping against open/close chatter (which wears STS3215 gears). At 0.0 you get visible oscillation; make it too negative and the policy learns never to close.
τ_{succ}	<code>env. processor. reward_ classifier. success_ threshold</code>	0.5	not used in sim (env gives reward)	0.7 to 0.8	Probability above which the classifier emits <code>success_reward</code> and (with <code>terminate_on_success</code>) ends the episode. 0.5 maximises accuracy but false positives are far more damaging than false negatives: a false positive teaches the critic that a failed state is terminal-good. Bias towards precision.

Part B: Failure-mode register, ranked

B1. Learner dies before step 0 with a draccus `TypeError`. *Symptom:* `learner.py:166` calling `cfg.to_dict()` raises inside `isinstance(x, typing.Any)`. *Cause:* `src/lerobot/envs/configs.py:284` annotated `fixed_reset_joint_positions: Any | None`, which draccus cannot encode. *Fix:* the annotation must be `list[float] | None`. This was fixed upstream in PR #4297. Checkouts pinned at or before v0.6.1 commit `2aba372b` still carry the bug and need the annotation changed locally; verify with `grep -n fixed_reset_joint_positions src/lerobot/envs/configs.py` before assuming either way.

B2. Out of memory during the first few hundred transitions. *Symptom:* CUDA OOM, or the host swapping and the learner dropping to <1 Hz. *Cause:* `ReplayBuffer._initialize_storage` (`src/lerobot/rl/buffer.py:147`) calls `torch.empty(...)` with no `dtype`, so images are float32. Two cameras at $3 \times 128 \times 128$ is 384 KiB per state, doubled because `next_states` is stored separately, so 100000 capacity is ~ 75 GiB. *Fix:* keep `storage_device: "cpu"`, cut capacity to what your session actually produces, and if you need more, store uint8 or enable the buffer's memory-optimised path.

B3. `udev` name churn between sessions. *Symptom:* the follower connects as the leader, or a camera index swaps and the wrist view is now the front view. *Cause:* `/dev/ttyACM*` and `/dev/video*` are assigned in enumeration order. *Fix:* write udev rules keyed on serial number to create stable symlinks, and pin camera identity by `/dev/v4l/by-id/`. A swapped camera is silent: the policy just stops working.

B4. USB bandwidth exhaustion. *Symptom:* dropped frames, `fps` falling below 10, `select timeout` from a camera, jerky control. *Cause:* two uncompressed UVC streams plus the servo bus on one root hub exceeds the available bandwidth. *Fix:* put the cameras on separate host controllers, request MJPG rather than YUYV, and keep capture resolution at the 128×128 the policy consumes rather than downscaling from $1080p$.

B5. Reset pose mismatch. *Symptom:* the first observation of every episode is out of distribution; success rate is high mid-episode but the policy fails from reset. *Cause:* `fixed_reset_joint_positions` does not match the pose the demos were recorded from, or the arm was recalibrated between recording and training. *Fix:* record the reset pose alongside the dataset, and re-verify calibration ids `follower_arm` / `leader_arm` before any session that reuses old data.

B6. Workspace bounds set too large. *Symptom:* long stretches of episodes where the EE is nowhere near the object, reward is identically zero, and the critic flattens. *Cause:* `end_effector_bounds` covering a volume the agent cannot usefully explore in `control_time_s` at 10 Hz. *Fix:* shrink the box to just contain the demonstrated trajectories plus a margin. HIL-SERL's sample efficiency comes from a small, well-posed state space.

B7. Long human interventions destroy the Q function. *Symptom:* after a rescue lasting several seconds, critic loss spikes and the autonomous policy is worse than before. *Cause:* long off-policy segments are far outside the current policy's support, so the entropy-augmented backup $y = r + \gamma(1 - d) \left(\min_i Q_{\bar{\theta}_i}(s', a') - \alpha \log \pi_{\phi}(a'|s') \right)$ is evaluated at actions the actor would never take, and the bootstrapped value is unreliable. *Fix:* intervene briefly and often rather than rarely and long; nudge the arm back onto a recoverable state and release.

B8. Reward classifier is confidently wrong. *Symptom:* validation accuracy 97%, real-world reward nonsense. *Cause:* class imbalance (successes are a few frames per episode, so predicting "fail" scores ~95%) and temporal leakage (adjacent frames from the same episode split across train and validation, so the model memorises the scene rather than the outcome). *Fix:* split by **episode**, never by frame; report precision/recall on the positive class, not accuracy; balance or reweight; and hold out episodes recorded on a different day.

B9. STS3215 thermal cutout. *Symptom:* mid-session, around 500 to 750 episodes, a joint goes limp and stops responding, then recovers minutes later. *Cause:* servo internal temperature reaching roughly 70 C triggers protection. *Fix:* log servo temperature every episode, schedule cooling breaks, reduce holding torque by choosing a reset pose that is closer to gravity-neutral, and treat a cutout as an episode to discard rather than a failure to learn from.

B10. Actor hangs on restart. *Symptom:* you restart the actor and it blocks forever without connecting. *Cause:* see Chapter 6. `learner_service.py:42` sets `MAX_WORKERS = 3`, exactly the number of long-lived RPCs one actor opens, and `StreamParameters` never checks `context.is_active()`, so a half-open connection from the dead actor pins all three worker threads permanently. *Fix:* restart the learner too, every time. Waiting does not help, because the threads are held by half-open RPCs rather than by socket state.

B11. Lighting drift. *Symptom:* success rate degrades smoothly over an afternoon with no config change. *Cause:* daylight through a window, plus camera auto-exposure and auto-white-balance chasing it, moves the visual input off the frozen ResNet10's training distribution. *Fix:* lock exposure, gain and white balance in v4l2; use controlled artificial lighting; and record the time of day per run so you can test this hypothesis later.

Part C: Experimental practice

Reproducibility: be honest about what is impossible

`seed: 1000` in `train_config.json` fixes network initialisation, the offline dataset iteration order, and the pseudo-random draws in sampling. It does not fix the experiment. Three things break exact reproducibility, and no amount of seeding recovers them.

First, actor and learner are separate processes exchanging parameters over gRPC on a wall-clock schedule (`policy_parameters_push_frequency` is in seconds). Which policy version generated transition t depends on scheduler contention, so two runs with identical seeds diverge within seconds. Second, the ratio of gradient steps to environment steps is emergent, not configured: the learner runs at whatever rate it achieves (~6-7 Hz observed), the actor runs at 10 Hz. Third, and decisively, you are in the loop. You will never intervene identically twice, and the hardware itself drifts (servo temperature changes friction, lighting changes pixels).

So do not claim reproducibility. Claim **replicability of the distribution**. Run at least five seeds, report median and interquartile range rather than a best curve, and pre-register a fixed evaluation protocol: a written list of initial object poses, a fixed number of trials, no interventions, success judged by a human against a written criterion. Report the evaluation protocol in the paper, not just the training curve.

What to log

The SAC algorithm already emits `stats.losses`, `stats.grad_norms` and `stats.extra["temperature"]`; send all of it to WandB (project `hilserl-sol01`). Add, per episode: return, episode length, terminated-vs-truncated, classifier success probability at termination, **and a separate human-entered success label**. Add the human-in-the-loop covariates: number of interventions, total intervention steps, fraction of actions in the buffer that were human. Add systems telemetry: optimization steps per second, actor steps per second, their ratio, buffer occupancy online and offline, policy staleness (learner step at push time minus learner step in use), servo temperatures, and dropped camera frames. The intervention rate over time is arguably your most important curve: if it is not decaying, the policy is not improving, whatever the return says.

Run structure and naming

One directory per run, named so that it sorts and greps: `sol01_pickplace_utd2_pushfreq2_seed1000_20260801T1430`. Set `job_name` to the same string. Record, inside the run directory: the resolved config (the same `cfg.to_dict()` that `learner.py:166` produces), the git SHA of your repo **and** of the vendored LeRobot submodule (v0.6.1 at 2aba372b, plus any local patch such as the `Any` fix), the dataset `repo_id` and revision, and the reward-classifier checkpoint hash.

This is what makes WandB config capture pay for itself. When you ask "what changed between run 12 and run 13", you want to select both runs and diff the config dicts, and get back `utd_ratio: 2 -> 4` rather than a memory. Every hyperparameter must reach WandB through the config object, never as an undeclared code edit. If you must patch LeRobot, patch it in the submodule and let the SHA record it.

Ablations worth the robot time

Robot time is your budget, so choose ablations that discriminate between explanations.

- **`use_backup_entropy` true vs false.** This removes the $-\alpha \log \pi_\phi(a'|s')$ term from the target. It tests whether entropy in the backup is what stabilises learning here, or merely decorates it.
- **SiLU vs tanh in the MLP trunks.** The default activation is `nn.SiLU()` (`modeling_gaussian_actor.py:327`); `critic_network_kwargs` and `actor_network_kwargs` let you change it. Cheap, and a common reviewer question.
- **`utd_ratio` 1 vs 2 vs 4, reported twice.** Once step-matched (same environment steps) and once wall-clock-matched (same minutes of robot time). These give different answers, and on hardware the wall-clock comparison is the honest one.
- **With vs without interventions.** Same demos, same everything, human hands off. This isolates the contribution of the "HIL" in HIL-SERL and is the single most informative ablation you can run.
- If time remains, `online_ratio` 0.5 vs 1.0, which tests how much the offline demos still matter late in training.

Reporting human-in-the-loop results honestly

The human is part of the system, so the human belongs in the methods section. State who operated (and that the operator was the author, if so), how many hours of prior practice they had, and how interventions were triggered. Report total human wall-clock time as a cost alongside environment steps: a policy that reaches 90% in 20 minutes with 15 minutes of human effort is a different claim from one that does it with 2 minutes.

Report the intervention-rate curve. Report classifier-labelled success and human-labelled success separately, and say plainly that the classifier is an optimistic proxy (B8 explains why). Make the headline number an autonomous evaluation with no human present, fixed initial states, and a stated *n*. And report the runs that failed, including the ones killed by B9 and B11: in a field where single-run demonstration videos are common, a failure register is a contribution.

Further Reading

The four papers this book is built on

Haarnoja, Zhou, Abbeel and Levine (2018), "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor", ICML. arXiv:1801.01290. The origin of the objective derived in Chapter 2 and the algorithm dissected in Chapter 3. Read Sections 3 and 4 alongside the derivation here; the notation in this book was chosen to match it wherever possible.

Haarnoja et al. (2018), "Soft Actor-Critic Algorithms and Applications". arXiv:1812.05905. The follow-up that introduces the automatically tuned temperature α via the constrained formulation with target entropy $\tilde{H}$. This, not the ICML paper, is the version LeRobot actually implements: see the temperature loss at `src/lerobot/rl/algorithms/sac/sac_algorithm.py:419`.

Ball, Smith, Kostrikov and Levine (2023), "Efficient Online Reinforcement Learning with Offline Data" (RLPD), ICML. arXiv:2302.02948. The justification for symmetric sampling from an offline demonstration buffer and an online buffer, which appears in the config as `mixer: "online_offline"` with `online_ratio: 0.5`, and for the high update-to-data ratio that `utd_ratio` controls. Chapter 4 leans on this paper heavily.

Luo, Xu, Wu, Levine et al. (2024), "Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning". arXiv:2410.21845. HIL-SERL itself: the human intervention mechanism, the treatment of interventions as off-policy data, and the real-robot results that motivate the entire practical half of this book.

Luo, Hu, Xu, Tan and Levine (2024), "SERL: A Software Suite for Sample-Efficient Robotic Reinforcement Learning", ICRA. arXiv:2401.16013. The predecessor system. Useful mainly for the engineering decisions it documents: the actor/learner split, the reward classifier, and the discipline around reset behaviour that reappears in `ResetConfig`.

Useful supporting papers

- Haarnoja, Tang, Abbeel and Levine (2017), "Reinforcement Learning with Deep Energy-Based Policies". arXiv:1702.08165. Where the soft Bellman operator comes from, if the Chapter 2 derivation leaves you wanting the energy-based view.
- Fujimoto, van Hoof and Meger (2018), "Addressing Function Approximation Error in Actor-Critic Methods" (TD3). arXiv:1802.09477. The origin of the clipped double-Q trick that `num_critics: 2` and the `min_i` in the TD target implement.
- Chen, Wang, Zhou and Ross (2021), "Randomized Ensembled Double Q-Learning: Learning Fast Without a Model" (REDQ). arXiv:2101.05982. Why a high update-to-data ratio needs an ensemble to stay stable, which is the argument behind `utd_ratio` and `num_subsample_critics`.
- Ross, Gordon and Bagnell (2011), "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning" (Dagger). arXiv:1011.0686. The classical framing of the distribution-shift problem that human intervention solves in a different way.

Documentation and code

- HIL-SERL on real hardware: <https://huggingface.co/docs/lerobot/hilserl>
- HIL-SERL in simulation: https://huggingface.co/docs/lerobot/hilserl_sim
- Reference configs (the source of every default quoted in this book): https://huggingface.co/datasets/lerobot/config_examples/, specifically `rl/gym_hil/train_config.json`
- SO-101 / SO-ARM100 hardware, URDF and assembly: <https://github.com/TheRobotStudio/SO-ARM100>
- LeRobot source: <https://github.com/huggingface/lerobot>, pinned here at v0.6.1, commit `2aba372b`
- Local copy of the tutorial that matches the pinned code: `vendor/lerobot/docs/source/hilserl.mdx`. When the rendered documentation and the local `.mdx` disagree, the local file is the one that matches your

checkout.

Reading order suggestion

If you are coming to this cold, read Chapters 1 to 3 of this book first, then Haarnoja 2018b, then Chapter 4, then RLPD. Save the HIL-SERL paper until after you have a simulated run producing a learning curve: it is a systems paper, and it reads very differently once you have felt the failure modes it is quietly designed around.